Training Copilot: A Multi-Agent Sports Analytics Platform Using the Model Context Protocol

Seminar Paper
AI in Service Systems II

Group 6

Maximilian Klasen, Aaron Neumann, Lorenz Neugebauer,
Marvin Janzen, Simon Meyer

At the Department of Economics and Management

Digital Service Innovation (DSI)

Karlsruhe Digital Service Research & Innovation Hub (KSRI) &
Institute for Information Systems (WIN)

Examiner:	Prof. Dr. Gerhard Satzger
Second Examiner:	Prof. Dr. Hansjörg Fromm
Date of Submission:	August 17, 2026

Declaration of Academic Integrity

We hereby confirm that the present work is solely our own work and that if any text passages or diagrams from books, papers, the Web or other sources have been copied or in any other way used, all references—including those found in electronic media—have been acknowledged and fully cited.

Karlsruhe, August 17, 2026

Maximilian Klasen, Aaron Neumann, Lorenz Neugebauer, Marvin Janzen, Simon Meyer

Abstract

Recreational and competitive athletes rely on multiple digital platforms to track training load, monitor recovery, plan routes, and schedule sessions. However, this data remains fragmented across proprietary ecosystems—Strava for activities, Garmin for wellness, weather services for planning, and calendars for availability—forcing users into manual cross-referencing that is time-consuming and error-prone. This paper presents *Training Copilot*, an open-source, multi-agent sports analytics platform that unifies these heterogeneous data sources behind a single conversational interface. Training Copilot adopts the Model Context Protocol (MCP) as a uniform integration layer: each external API is encapsulated in an independent MCP server, and a central host discovers and routes tool calls without hardcoding provider-specific logic. On top of this data layer, a LangGraph-based multi-agent system coordinates five specialist agents—recovery, training load, environmental context, route planning, and fitness knowledge—via the Agent-to-Agent (A2A) protocol. Each specialist is an autonomous Reasoning-and-Acting (ReAct) agent scoped to its own MCP servers. An orchestrator agent decomposes user queries, delegates to the appropriate specialists in parallel, and synthesises data-driven recommendations. The fitness knowledge specialist employs Retrieval-Augmented Generation (RAG) over a curated vector database of public-domain exercise science literature, grounding its advice in verifiable sources. We evaluate the system through an automated end-to-end assessment using MLflow's conversation simulation framework, employing ten synthetic personas across two user archetypes that collectively exercise all five specialist domains. Each conversation is scored along five dimensions—conversation completeness (whether the user's goal was achieved), user frustration (resilience under adversarial follow-ups), safety (absence of harmful advice), supportive coaching tone (empathy under pushback), and data grounding—where the last scorer is fully deterministic, verifying tool usage from execution traces rather than from chat text. The MCP-based architecture enables seamless extensibility—adding a new data source requires exactly one file and one configuration line—while the multi-agent design produces contextually aware, data-grounded training recommendations that integrate recovery status, workload trends, weather conditions, and route options into a single coherent response.

Contents

Acronyms

A2A	Agent-to-Agent (protocol)
ACWR	Acute-to-Chronic Workload Ratio
API	Application Programming Interface
ATL	Acute Training Load
BMI	Body Mass Index
CAGR	Compound Annual Growth Rate
CoT	Chain-of-Thought
CTL	Chronic Training Load
EMA	Exponential Moving Average
GPS	Global Positioning System
HR	Heart Rate
HRV	Heart Rate Variability
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
LLM	Large Language Model
MCP	Model Context Protocol
MFA	Multi-Factor Authentication
NLP	Natural Language Processing
OAuth	Open Authorization
ORS	OpenRouteService
OSM	OpenStreetMap
RAG	Retrieval-Augmented Generation
ReAct	Reasoning and Acting
REST	Representational State Transfer
RPC	Remote Procedure Call
SBERT	Sentence-BERT
SpO_2	Peripheral Oxygen Saturation
SSE	Server-Sent Events
SSRF	Server-Side Request Forgery
TRIMP	Training Impulse
TSB	Training Stress Balance
TSS	Training Stress Score
UI	User Interface
UV	Ultraviolet
VO_2max	Maximal Oxygen Uptake

List of Figures

List of Tables

1 Introduction

Wearable devices from Garmin, Polar, and Apple capture heart rate, GPS trajectories, sleep stages, heart-rate variability (HRV), and stress levels continuously (Seckin, Ates, & Seckin, 2023), and Strava alone aggregates activity data from over 180 million users (Strava, 2025b). Yet this unprecedented volume of training data remains fragmented across proprietary ecosystems with incompatible interfaces, making holistic analysis impractical for anyone without sports-science training (Bourdon et al., 2017). An endurance cyclist we interviewed during user research illustrates the problem concretely: after every ride he exports his Strava data and uploads it to ChatGPT for analysis—a process that is slow and incomplete, since the general-purpose model has no access to his Garmin recovery data, the weather forecast, or his calendar. Deciding whether to train on a given day instead requires checking a Garmin watch for HRV and sleep quality, Strava for cumulative load, a weather service for conditions, and a calendar for scheduling, with the integration across these four applications left entirely to the athlete.

Large language models have demonstrated the ability to reason across heterogeneous information sources once equipped with tool-calling capabilities (Qu et al., 2025; Schick et al., 2023), and the emergence of open interoperability protocols—Anthropic's Model Context Protocol (MCP) (Anthropic, 2024b) for tool discovery and invocation, and Google's Agent-to-Agent protocol (A2A) (Google Cloud, 2025) for inter-agent coordination—makes it newly feasible to connect these fragmented sources behind a single conversational interface.

1.1 Problem Statement and Approach

This paper presents *Training Copilot*, an open-source sports analytics platform addressing three shortcomings of the current landscape. *Data fragmentation*: training and wellness data is distributed across platforms with incompatible authentication models, formats, and access patterns, and although sports-science consensus statements emphasise that effective load monitoring requires combining external and internal load from multiple sources (Bourdon et al., 2017; Halson, 2014), the tools that collect this data provide no unified view. Training Copilot solves this with MCP as a uniform data layer: each external API is encapsulated in an independent MCP server, and a single host client discovers and routes tool calls without hardcoding provider-specific logic. *Lack of cross-domain reasoning*: platforms such as Strava, Garmin Connect, Intervals.icu, and TrainingPeaks display metrics in isolation—a sleep score here, a training-load chart there—even though an athlete's readiness to

train depends on the interplay of recovery status, cumulative workload, environmental conditions, and schedule (Bourdon et al., 2017; Gabbett, 2016). Training Copilot addresses this with a multi-agent system: five LangGraph (LangChain, 2024) ReAct (Yao et al., 2023) agents, each scoped to a domain (recovery, load, context, routes, fitness knowledge), coordinated by an orchestrator via A2A. *Rigid architectures*: existing platforms are tightly coupled to their data sources, so adding one (e.g. a nutrition tracker) requires code changes throughout the stack; Training Copilot's MCP-based design reduces this to one server file and one configuration line, consistent with established microservice design principles (Dragoni et al., 2017).

1.2 Contributions

- An MCP-based integration architecture achieving single-line extensibility, verified empirically by adding a Telegram bridge and a flythrough renderer without changes to the host, agents, or UI.
- A multi-agent system with five domain-scoped specialists coordinated via A2A; scoping each specialist to at most three MCP servers counters the tool-selection degradation observed with large tool sets (Qu et al., 2025).
- A RAG-based fitness knowledge agent that grounds exercise-science advice in a local vector database of public-domain literature, with explicit citation preservation.
- An automated evaluation framework built on MLflow conversation simulation (MLflow, 2026), using ten persona-driven test cases and five scoring dimensions, including a deterministic trace-based grounding scorer.

1.3 Structure of This Paper

Section 2 analyses the sports analytics market and the gap Training Copilot addresses; Section 3 reviews the state of the art in agentic AI, tool-integration protocols, sports analytics, and retrieval-augmented generation; Section 4 presents the system architecture; Section 5 describes the data sources; Section 6 details agent interaction patterns; Section 7 describes the evaluation methodology; Section 8 discusses trade-offs, limitations, and ethics; and Section 9 closes with a summary and future work.

2 Market Analysis

This section examines the sports-analytics and fitness-technology market, identifies the competitive landscape, and articulates the gap Training Copilot addresses, closing with a personal account of what motivated the work.

2.1 Market Size and Growth

The global sports-analytics market is driven by the convergence of wearable adoption, cloud computing, and consumer-facing large language models. Grand View Research estimates it at USD 5.7 billion in 2025, projecting USD 23.1 billion by 2033 at an 18.5 % CAGR (Grand View Research, 2026), alongside a large and steadily growing wearable-device market (IDC, 2024). Demand is particularly pronounced in Germany, where fitness-app downloads rose from 41.37 million in 2017 to a projected 135.22 million in 2025 (Statista, 2025). Strava's 2025 trend report—based on 30,494 respondents drawn from its 180-million-user community and a random non-user sample—finds that 30 % of Generation Z plan to increase fitness spending in 2026 and that 46 % of all respondents would use AI as a smart coach for sports (Strava, 2025b). This converges with the broader generative-AI boom: McKinsey estimates USD 2.6–4.4 trillion in annual cross-industry value, concentrated in knowledge-intensive domains where synthesising heterogeneous information is the core task (McKinsey & Company, 2023)—a description that fits sports coaching and wellness precisely, where the athlete must integrate physiological data, environmental conditions, and scheduling constraints.

2.2 Competitive Landscape

Table 1 summarises the platforms athletes currently use to manage training data and their key limitations.

WHOOP launched WHOOP Coach (WHOOP, 2024), an OpenAI-powered conversational assistant that answers questions about the user's biometrics but sees only WHOOP data, with no access to Strava load, weather, or calendar context; Athletica.ai (Athletica.ai, 2024) dynamically adjusts endurance training plans to an athlete's fitness, fatigue, and schedule but offers no conversational interface or cross-domain reasoning. Both represent meaningful progress, yet each remains siloed within its own ecosystem and offers either conversation *or* adaptive planning, never both grounded in multi-source data simultaneously.

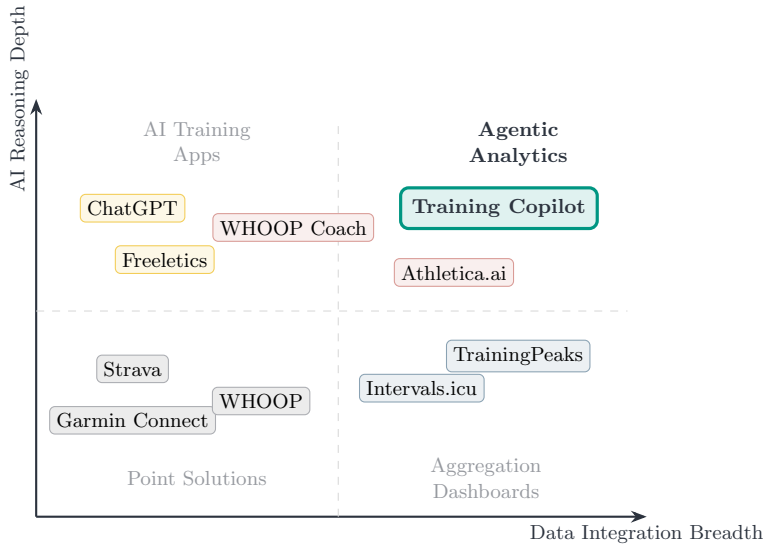

Fig. 1 Solution landscape. Point solutions cover single data domains; aggregation dashboards combine data but lack reasoning; AI training apps reason but lack data access. Training Copilot combines broad integration with deep agentic reasoning.

A wave of AI features launched in 2025 sharpens rather than closes this gap. Garmin introduced Garmin Connect+ (Garmin, 2025), a premium tier whose "Active Intelligence" surfaces AI-generated insights, and Oura released Oura Advisor (Oura, 2025), a conversational LLM health coach with per-user memory—but both reason only over their own device's data, exactly the walled-garden limitation Training Copilot targets. On the planning side, Strava acquired the running-plan app Runna (Strava, 2025a) and dedicated engines such as TriDot (TriDot, 2025) deliver AI-driven, data-optimised triathlon plans; these adapt training effectively, but through a fixed optimisation engine rather than a conversational interface that lets an athlete interrogate their live weather, calendar, and recovery on demand. Across the 2025 cohort the pattern holds: conversational products stay single-source, and adaptive planners stay non-conversational. General-purpose assistants are the exception that proves the rule—ChatGPT, Gemini, and Claude converse fluently but, used directly, see none of the athlete's data. That boundary is only beginning to blur: Strava has announced an official MCP server (Strava, 2026), still rolling out rather than generally available, that would let an MCP-native assistant such as Claude query a user's Strava data directly—a first step that nonetheless remains single-source and validates precisely the protocol-first integration Training Copilot already delivers across all its sources (Section 5.1).

The landscape can also be read along two axes—breadth of data integration and depth of AI-driven analysis (Figure 1).

Point solutions (Garmin, Strava, weather apps) each cover a single domain; aggregation dashboards (Intervals.icu, TrainingPeaks) combine sources but apply no agentic reasoning; AI-based training apps typically lack access to the user's actual wearable data. Training Copilot targets the upper-right quadrant: broad integration combined with deep, agentic reasoning. No platform integrates all five dimensions that matter for training decisions—activity history, wellness, weather, calendar, and routes; the platforms with analytical depth (TrainingPeaks, Intervals.icu) operate purely as dashboards that visualise without reasoning across sources; and general-purpose LLMs such as ChatGPT can converse but have no access to the athlete's actual data, producing generic advice that ignores individual context (Puce, Bragazzi, Currà, & Trompetto, 2025).

2.3 The Market Gap

We identify a gap at the intersection of three capabilities that no existing consumer product occupies simultaneously (Figure 2): *multi-source data integration*—unifying wearables (Garmin), activity platforms (Strava), environmental services (weather), productivity tools (Google Calendar), and geographic services (OpenRouteService) through a single interface; *contextual, cross-domain reasoning*—synthesising recovery status, load trends, weather, and schedule into one coherent recommendation rather than isolated metrics (Bourdon et al., 2017); and *conversational interaction*—a natural-language interface for complex, multi-faceted questions answered from the athlete's own live data.

2.4 Target Users

Two user archetypes emerge from this analysis and also inform our evaluation design (Section 7). The *ambitious endurance athlete* trains in structured blocks, tracks Training Stress Balance obsessively, wears a Garmin watch for HRV and recovery, and wants precise, data-driven go/no-go decisions for key sessions—frustrated by having to cross-reference Strava trends with Garmin recovery data manually, with no platform integrating weather and calendar into the recommendation. The *recreational cyclist* rides for enjoyment and social motivation, cares about scenic routes and weather comfort, finds traditional dashboards overwhelming, and wants plain-language answers ("Is tomorrow good for a ride?", "Find me a nice 40 km loop") without jargon. These archetypes anchor opposite ends of the analytical-depth spectrum, so Training Copilot must serve both data-hungry power users and casual athletes who prefer plain language.

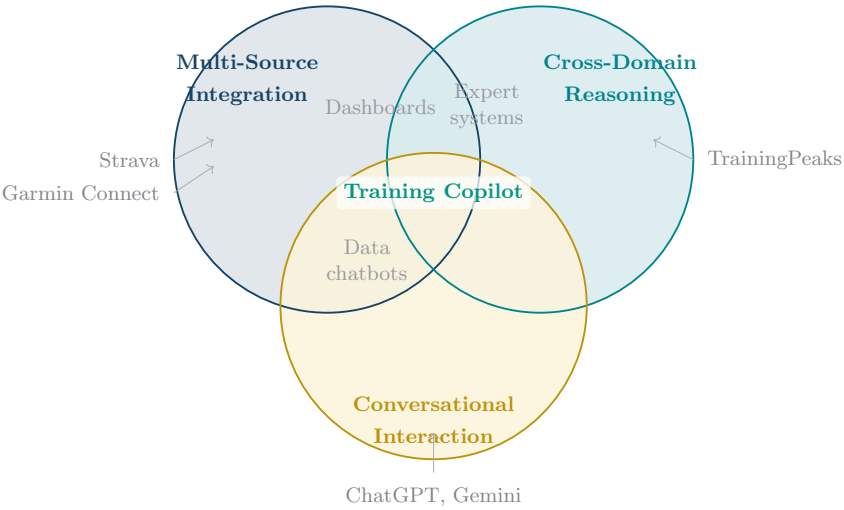

Fig. 2 The Training Copilot market gap. Existing platforms cover at most two of the three capabilities. Training Copilot is positioned at the intersection of all three, combining multi-source data integration (MCP layer), cross-domain contextual reasoning (multi-agent system), and conversational interaction (chat interface).

2.5 Personal Motivation

The following reflects the author's personal perspective rather than an objective market assessment.

The motivation for Training Copilot emerged from direct frustration as an endurance athlete: answering "Should I do my long run today or push it to Thursday?" meant checking five applications—Garmin for HRV and sleep, Strava for load trends, a weather service, Google Calendar, and sometimes a route planner—and mentally synthesising conflicting signals. The platforms with the data offered no reasoning; the tools that could reason (ChatGPT, Gemini) had no data access. The emergence of MCP and A2A made it feasible to build the system we wished existed: one that treats this as a single question rather than five separate lookups.

Table 1 Competitive landscape of consumer sports analytics platforms. No existing platform simultaneously provides multi-source integration, cross-domain reasoning, and conversational interaction.

Platform	Core Capability	Data Sources	Key Limitation
Strava	Activity tracking, social, routes	Strava uploads	No recovery data, no weather, no cross-source reasoning
Garmin Connect	Wellness, sleep, HRV, Body Battery	Garmin only	Walled garden; no Strava load, no route planning
Garmin Connect+	AI "Active Intelligence" insights (premium)	Garmin only	Push insights, not conversational; single ecosystem, no cross-source reasoning
TrainingPeaks	Training plans, TSS/CTL/ATL	File import	Paid, no chat interface, manual import
Intervals.icu	Load analytics, free tier	Strava, Garmin	No weather, calendar, or routes; dashboard-only
WHOOP Coach	AI chat on biometrics	WHOOP device	Walled garden; no Strava, routes, or calendar
Oura Advisor	Conversational AI health coach (LLM)	Oura ring only	Wellness only; no training load, weather, or routes
Athletica.ai	Adaptive training plans	Garmin, Strava	Plan-only; no recovery chat, no route planning
Runna	Personalised running training plans	Runner profile, Strava sync	Plan generation only; no conversational, cross-domain reasoning
TriDot	AI/FitLogic triathlon optimisation	Athlete training data	Optimisation engine, not a conversational cross-domain assistant
ChatGPT / Gemini / Claude	General-purpose LLM	None used directly (Strava MCP rolling out)	No live fitness data out of the box; generic, ungrounded advice

3 State of the Art

This section reviews the four research pillars Training Copilot builds upon: agentic AI and multi-agent systems, tool-integration protocols, sports analytics and wearable technology, and retrieval-augmented generation.

3.1 Agentic AI and Multi-Agent Systems

The rapid scaling of large language models has shifted the field from static text generation toward autonomous *agentic* systems that plan, reason, and act in external environments. The Transformer architecture (Vaswani et al., 2017), chain-of-thought prompting (Wei et al., 2022), and the scaling of model capacity to hundreds of billions of parameters (Brown et al., 2020) laid the groundwork by showing that LLMs perform multi-step reasoning once prompted to generate intermediate steps, with subsequent frontier models reaching human-level performance across professional and academic benchmarks (OpenAI, 2023). We organise the literature along three themes: reasoning architectures, agent-internal structure, and multi-agent coordination.

3.1.1 Reasoning Architectures

Two complementary patterns ground LLM reasoning in external action. Yao et al. (2023) introduced *ReAct*, interleaving reasoning traces ("thought" steps) with task-specific actions in a single loop: the model generates a thought, executes an action (e.g. `search[entity]` against a Wikipedia API), observes the result, and repeats until done. Evaluated on PaLM-540B, ReAct reached 71 % success on ALFWorld (vs. 45 % action-only) and 40 % on WebShop (vs. 30.1 %), and produced zero hallucinated reasoning chains on HotpotQA/FEVER versus 56 % for chain-of-thought—because every claim is grounded in a retrieved observation rather than generated from parameters. Training Copilot's specialists use this pattern directly: each is a LangGraph ReAct agent whose "actions" are MCP tool calls. Shinn et al. (2023) extended the loop across episodes with *Reflexion*, where agents store linguistic self-reflections after failures in an episodic buffer to inform later trials without weight updates—complementary to ReAct's within-episode loop. We use Reflexion in one subsystem: the LLM-driven chart generator retries once with the error message as feedback (Section 4.7). Both patterns, however, assume a single agent over a fixed tool set; neither addresses multi-agent coordination or dynamic tool discovery across heterogeneous sources.

3.1.2 Agent-Internal Architecture

Park et al. (2023) showed that LLM-powered agents exhibit believable behaviour when equipped with a *memory stream* recording experiences, a *reflection* mechanism synthesising abstractions, and a *planning* module translating reflections into action sequences; Wang et al. (2024) generalised this into a four-module taxonomy—profiling, memory, planning, action—subsuming prior architectures. Both works find memory and planning mutually reinforcing (richer memory enables better planning, which yields more informative experience), but both describe single-agent architectures and do not address how agents with different profiles and tool sets should coordinate.

3.1.3 Multi-Agent Coordination

Three works illustrate the multi-agent design space. Shen et al. (2023) proposed HuggingGPT, where a central controller decomposes requests, selects specialist models from a hub, executes them, and summarises results—demonstrating orchestrator–specialist decomposition, though applied to model selection rather than tool-based data retrieval. Wu et al. (2024) introduced AutoGen, showing empirically that role-specialised agents with tool access outperform monolithic designs on complex tasks, and that flexible conversation patterns (sequential, parallel, nested) enable different coordination strategies. Guo et al. (2024) surveyed the field and identified recurring open challenges in agent orchestration, inter-agent communication, and scalability—while Acharya, Kuppan, and Divya (2025) distinguished agentic AI from traditional automation by its capacity for autonomous, goal-directed decision-making and adaptation in dynamic environments. These works converge on a shared finding: multi-agent architectures with domain-scoped specialists outperform monolithic agents on heterogeneous tasks, yet none demonstrate *standardised* tool-discovery or inter-agent protocols, relying instead on ad-hoc registries and custom communication layers.

3.2 Tool Integration Protocols

The practical utility of LLM agents depends on interacting with external tools and data. Research here progressed through three stages: teaching models to use tools at all, standardising how tools are discovered and invoked, and standardising how tool-using agents talk to each other.

3.2.1 Tool-Augmented Language Models

Schick et al. (2023) showed that a 6.7B-parameter model (GPT-J) can teach itself to use tools through a self-supervised pipeline: candidate API calls are sampled within training text, executed, and kept only if they reduce the cross-entropy loss over subsequent tokens. Trained on five tools (calculator, QA, Wikipedia search, translation, calendar), the model selected the correct tool in 97.9 % of cases on mathematics benchmarks, outperforming GPT-3 (175B) despite being 25× smaller—though limited to one tool call per example with no query reformulation. Patil, Zhang, Wang, and Gonzalez (2023) addressed tool selection at scale, fine-tuning LLaMA-7B on 16,450 instruction–API pairs across 1,645 APIs from three model hubs; on their APIBench benchmark, Gorilla beat GPT-4's zero-shot accuracy on TorchHub by 20.43 points and cut hallucinated API calls from 36.55 % (GPT-4) to 6.98 %, reaching 0 % on TorchHub with a ground-truth retriever. Qu et al. (2025) synthesised these findings into a four-stage workflow—planning, selection, calling, response generation—and showed across benchmarks scaling to 16,464 tools (ToolBench) that placing all tool descriptions in context becomes impractical beyond a few hundred tools, making retrieval-based pre-filtering mandatory. This motivates Training Copilot's design directly: scoping each specialist to at most three MCP servers (1–27 tools per specialist) keeps tool selection well below that threshold.

3.2.2 From Ad-hoc Integration to Standardised Protocols

The proliferation of tool-augmented agents created an $M \times N$ fragmentation problem—M applications each integrating N data sources separately. Two complementary protocols address this. The **Model Context Protocol (MCP)** (Anthropic, 2024a, 2024b) defines a universal client–server protocol over JSON-RPC 2.0 (JSON-RPC Working Group, 2013) for tool discovery and invocation, exposing Prompts, Resources, and Tools while separating authentication from tool declaration so a host can use arbitrary servers without provider-specific code. Jayanti and Han (2026) proposed Context-Aware MCP (CA-MCP), adding a *Shared Context Store* that servers read and write to after the central LLM seeds it with goals and constraints—reducing execution time by 67.8 % and LLM orchestration calls by 60 % on the TravelPlanner benchmark while improving task completeness. Training Copilot does not adopt this shared-memory pattern (A2A handles inter-agent coordination instead), but the result validates MCP's extensibility. The **Agent-to-Agent protocol (A2A)** (Google Cloud, 2025), donated to the Linux Foundation after Google's April 2025 announcement, standardises inter-agent communication over JSON-RPC 2.0 around *opaque internals*: agents advertise capabilities via Agent

Cards but expose only results, not reasoning, enabling composition across frameworks and providers. Yang et al. (2025) distinguish the two as complementary: MCP is *context-oriented* (agents to data), A2A is *coordination-oriented* (agents to agents). Both protocols are new enough that their security surface is still being charted: Hou, Zhao, Wang, and Wang (2025) give the first systematic analysis of MCP's threat model across the full server lifecycle (creation, deployment, operation, and maintenance), identifying tool poisoning and injection risks that any multi-tenant deployment must address—safeguards we scope out of this prototype and return to in Section 8.3. To the best of our knowledge, no published system has yet combined both protocols in a domain-specific multi-agent application.

3.3 Sports Analytics and Wearable Technology

Sports science converges on one finding central to this work: effective training management requires integrating multiple physiological and contextual signals, yet no consumer tool performs this integration with reasoning. We review three themes: training load and periodisation, recovery monitoring, and wearable reliability.

3.3.1 Training Load: Monitoring and Injury Prevention

Bourdon et al. (2017), in an international consensus statement (11 authors, five countries), formalise *external load*—objective work performed, measured by GPS and accelerometers—against *internal load*—the psychophysiological response (heart rate, HRV, session RPE, lactate)—and recommend monitoring both jointly, since identical external load produces different internal responses depending on fatigue or illness; this "uncoupling" is itself diagnostic. They establish the acute-to-chronic workload ratio (ACWR), with 0.8–1.3 as low-risk; Halson (2014) add that no single biomarker is sufficiently validated alone. Gabbett (2016) formalise the "training–injury prevention paradox" from cricket, Australian football, and rugby league data: well-conditioned athletes have *fewer* injuries than undertrained ones, since conditioning itself protects. At ACWR $\leq$0.99, seven-day injury likelihood was $\sim$4 % in elite fast bowlers; at $\geq$1.5, risk rose 2–4$\times$, and week-to-week load increases of 15 %+ tracked with 21–49 % injury rates versus under 10 % when load stayed roughly constant week to week. Grgic, Mikulic, Podnar, and Pedisic (2017) extend this to periodisation, finding in a meta-analysis that linear and undulating schemes produce equivalent hypertrophy—the specific periodisation pattern does not significantly affect outcomes. Together these findings establish that load management requires trend analysis over time, not point-in-time metrics; Training Copilot's load specialist computes CTL, ATL, and TSB from Strava data—derived from the fitness–fatigue impulse-response model of Morton, Fitz-Clarke, and Banister (1990)—to operationalise this. The ACWR itself,

however, has faced methodological criticism: Impellizzeri, McCall, Ward, Bornn, and Coutts (2020) identify mathematical coupling (acute load appears in both numerator and denominator) and show that simulations can produce spurious ACWR–injury associations from unrelated data, so our system presents ACWR as one contextual signal rather than a standalone predictor.

3.3.2 Recovery: From Single Metrics to Multi-Signal Assessment

Kellmann et al. (2018) argue in a consensus statement that systematic recovery monitoring—not just load tracking—should be integral to training, since the stress–recovery balance determines long-term sustainability. HRV has emerged as a central biomarker, but its interpretation is not straightforward: Plews, Laursen, Stanley, and Buchheit (2013), using data from Olympic and World Champion rowers, show that in elite athletes both increases *and* decreases in HRV can signal negative adaptation, because vagal HRV saturates at very low resting heart rates (the HRV–R-R interval relationship is quadratic, not linear). They recommend weekly rolling averages of Ln rMSSD over single-day values (which showed no meaningful performance correlation, $r = -0.17$) interpreted against each athlete's own baseline—among four gold-medal rowers, the Ln rMSSD–R-R correlation ranged from $r = 0.91$ to $r = -0.03$, profiles a fixed-threshold system would misread. Lundstrom, Foreman, and Biltz (2023) review HRV-guided training in consumer settings, noting the gap between clinical- and consumer-grade measurement. The strongest evidence for multi-signal approaches comes from Rothschild, Stewart, Kilding, and Plews (2024), who tracked 43 endurance athletes (67 % triathletes) over 12 weeks (3,572 person-days) and tested nine ML algorithms predicting daily perceived recovery status: the best group model (LASSO) reached $R^2 = 0.31$ (RMSE 11.8 on 0–100), individual top-5-feature models reached $R^2 = 0.35 \pm 0.20$ with RMSE varying fivefold (5.5–23.6) across participants, and the most predictive features were soreness, sleep index, and prior-day status—not HRV alone. Wackerhage and Schoenfeld (2021) frame this as inherent: a training plan is a complex mix of interacting, time-varying interventions, making purely evidence-based decisions impractical—motivating tool-assisted approaches like Training Copilot's recovery specialist, which queries HRV, sleep, Body Battery, and stress simultaneously to approximate this multi-signal assessment.

3.3.3 Wearable Reliability and Data Ecosystem Biases

Wearable data reliability is a prerequisite for any analytics system built on it. Fuller et al. (2020) reviewed 158 publications across 45 device models from nine manufacturers: of 979 step-count comparisons, 45 % of those with calculable error fell

within ±3 % of criterion measures (slight underestimation, median −2 %), and 57 % of heart-rate comparisons were within ±3 % (Apple Watch 71 %, Fitbit 51 %, Garmin 49 %); precision degraded during high-intensity exercise and in free-living settings, energy expenditure was inaccurate across all devices, and while interdevice reliability for steps was very strong, intradevice reliability was variable (mean $r = 0.58$). Seckin et al. (2023) note that translating raw sensor data into actionable coaching remains open, and Hooren, Goudsmit, Restrepo, and Vos (2020) identify challenges in feedback timing and injury-risk communication for real-time wearable feedback in running. Broader reviews of wearable performance devices in sports medicine (Li et al., 2016) and of machine-learning-driven biomechanical analytics (Alzahrani & Ullah, 2024) reach a compatible conclusion: sensing has outpaced interpretation, and the real bottleneck is turning multi-stream data into decisions. Venter, Gundersen, Scott, and Barton (2023) further quantify selection bias in Strava's crowdsourced data—overrepresenting recreationally active, male, and middle-aged (35–54) users—a bias professional sports avoid (Bourdon et al., 2017) but consumer platforms must acknowledge. Training Copilot mitigates this partially through multi-source corroboration (cross-referencing HRV, sleep, and Body Battery) but cannot fully compensate for sensor inaccuracy.

Across all three themes, sports science establishes that training decisions require integrating load trends, recovery signals, and contextual factors, yet the tools that *collect* this data do not *reason across* it—the gap is in synthesis, not availability.

3.4 Retrieval-Augmented Generation

Retrieval-Augmented Generation combines the factual precision of information retrieval with generative fluency. Lewis et al. (2020) introduced the foundational architecture—a BART-large generator (400M parameters) augmented with a Dense Passage Retriever (DPR) over a FAISS index of 21 million Wikipedia passages—in two variants: RAG-Sequence (one retrieved passage conditions the whole output) and RAG-Token (a different passage per token). RAG set state-of-the-art on three open-domain QA benchmarks (44.5 EM on Natural Questions, 45.5 on WebQuestions, 52.2 on CuratedTrec) and was judged more factual than BART in 42.7 % of human comparisons versus 7.1 %, with more diverse outputs (distinct tri-gram ratio 53.8 % vs. 32.4 %); critically, its retrieval index can be hot-swapped without retraining. The retrieval component builds on Karpukhin et al. (2020), whose dual-encoder DPR outperforms BM25 by 9–19 % in top-20 retrieval accuracy, and Reimers and Gurevych (2019)'s Sentence-BERT, which adapts BERT via a Siamese training objective for efficient cosine-similarity comparison—the `sentence-transformers` library built on this work provides Training Copilot's embedding model (`all-MiniLM-L6-v2`,

∼90 MB, running locally without an embedding API). Gao et al. (2024) survey the progression from Naïve RAG (retrieve-then-generate) through Advanced RAG (query rewriting, re-ranking) to Modular RAG (pluggable components); Training Copilot implements the naïve form—embed, retrieve by cosine similarity, generate with retrieved context—sufficient for its curated corpus of ∼32,000 chunks from fifty public-domain books.

3.5 AI-Assisted Sports Coaching

AI-assisted sports coaching is an emerging field where three limitations recur: insufficient access to the athlete's actual data, limited cross-domain reasoning, and single-agent architectures that do not scale to heterogeneous tool sets.

On the data-access and personalisation gap, Monteiro-Guerra, Rivera-Romero, Fernandez-Luque, and Caulfield (2020) reviewed 28 papers covering 17 real-time physical-activity coaching apps across seven personalisation dimensions: feedback was universal (17/17) and goal setting near-universal (15/17), but context-awareness appeared in only 3/17 and behavioural adaptation in only 2/17, with implementation details given for just one context-aware system. Luo, Aguilera, Lyles, and Figueroa (2021) reach a similar conclusion for conversational agents specifically—chatbots sustain engagement, but most lack wearable-data access—and Puce et al. (2025) confirm this in a systematic review of 10 studies on generative AI for exercise prescription: AI-generated plans adhere to foundational training principles but lack specificity, progression, and, most critically, real-time physiological feedback, with identical prompts sometimes yielding different outputs.

On architecture, the most directly relevant prior system is GPTCoach (Jörke et al., 2025), an LLM-based coaching chatbot evaluated at CHI 2025. It pairs GPT-4 with a prompt-chaining design (separate calls for dialogue-state classification, motivational-interviewing strategy, and tool-call decisions) and two tools querying three months of Apple HealthKit data via HL7 FHIR; in a 16-participant lab study it scored 4.8/5 on feeling supported and 4.5/5 on empathy, with 93 % MI-consistent behaviour by external coding. Yet GPTCoach is a single agent with two tools and one data source, integrates no weather, calendar, or route data, and was evaluated only on a single onboarding session. At the architectural level, Vemuri, Decker, Saponaro, and Dominick (2021) proposed SMART-ALEC, a multi-agent health-coaching system whose per-user agent runs two concurrent modules—an analyser (spanning planning, scheduling, learning, context analysis, and execution) and a communicator—prototyped as BeSmart (an SMS-based system) and load-tested with up to 250 simulated agents, plus a JIT intervention system on Apple Watch with customised decision trees; predating MCP/A2A and coordinating via ad-hoc

GPGP-style commitment exchange, it nonetheless validates the modular-specialist-with-central-coordinator philosophy that Training Copilot extends with standardised protocols and LLM-based reasoning in place of finite-state dialogues.

On the human–AI boundary, Barger (2025) ran a mixed-methods RCT ($N = 52$) comparing sessions where participants believed they were coached by AI (actually a human behind a live-motion avatar) versus openly by a human: working-alliance scores did not differ significantly ($M = 72.73$ vs. 74.50; $t(50) = -0.71$, $p = 0.48$), and participants in the AI condition used fewer impression-management behaviours and disclosed more sensitive topics—supporting a synergy model of narrowly focused AI coaches working alongside humans. Gabarron, Larbi, Rivera-Romero, and Denecke (2024) survey AI-driven solutions for physical activity more broadly, finding recommender systems most studied, followed by conversational agents, with sparse long-term evidence for either.

In summary, prior coaching systems have either data access (wearable-native apps, limited to one source) or reasoning capability (general-purpose LLMs, without live data), but none combine multi-source integration, cross-domain reasoning, and literature-grounded advice via standardised protocols.

3.6 Summary and Research Gap

Each research stream reviewed above has matured individually, yet all share one limitation: isolation from the complementary capabilities integrated training support requires. Agentic AI has shown that domain-scoped multi-agent architectures outperform monolithic designs, but relies on ad-hoc registries rather than standardised protocols; MCP and A2A solve the resulting $M \times N$ integration problem in principle but have not been combined in a domain-specific application; sports science has established that training decisions require synthesising load, recovery, and context, yet consumer platforms display these signals without reasoning across them; AI-coaching systems demonstrate that LLM-based guidance is perceived as personalised and actionable but trade data access off against reasoning capability; and RAG enables grounding in verifiable sources but has not been applied to sports coaching alongside live physiological data.

Table 2 makes this concrete, scoring the most directly comparable prior systems—academic multi-agent frameworks, the closest published MCP extension, and the leading commercial AI coaching products—against four capabilities (multi-agent coordination, standardised protocols, multi-source live data, retrieval-grounded knowledge) and the application domain each targets. No prior system combines all four, and none of the few sports/health systems combine more than one.

Table 2 Comparison of prior agentic and coaching systems against the research-gap dimensions identified in this section. ✓ = demonstrated; × = not demonstrated or not present; parenthesised text qualifies a partial or negative result.

System	Multi-Agent	Protocol	Live Data	RAG-Grounded	Domain
HuggingGPT (Shen et al., 2023)	✓	× (ad-hoc)	× (model hub)	×	×
AutoGen (Wu et al., 2024)	✓	× (ad-hoc)	× (generic)	×	×
CA-MCP (Jayanti & Han, 2026)	× (1 LLM, reactive)	✓ (MCP extension)	× (benchmark sandbox)	×	× (travel/logistics)
GPTCoach (Jörke et al., 2025)	× (single agent)	×	× (HealthKit only)	×	✓
SMART-ALEC (Vemuri et al., 2021)	✓ (ad-hoc, per-user)	×	× (SMS/watch only)	×	✓
WHOOP Coach (WHOOP, 2024)	×	×	× (WHOOP only)	×	✓
Athletica.ai (Athletica.ai, 2024)	×	×	✓ (adaptive plans)	×	✓
General-purpose LLM (Puce et al., 2025)	×	×	× (no API access)	×	× (generic)
Training Copilot (this work)	✓	✓ (MCP + A2A)	✓ (5 live sources)	✓	✓

The research gap is therefore not in any individual capability but in their integration: no existing system combines standardised multi-source access (MCP), multi-agent coordination (A2A), domain-scoped specialist reasoning (LangGraph ReAct), and retrieval-grounded knowledge (RAG) for athlete-facing training support. Training Copilot's architecture maps each limitation to a specific design decision, detailed in Section 4.

4 System Architecture

This section presents Training Copilot's three-layer architecture: the MCP data layer, the agent layer, and the LLM integration seam. The guiding principle is *tool-agnostic extensibility*—no component names a specific tool or data source. Tools are discovered at runtime, servers are registered declaratively, and the LLM provider is resolved from configuration, so the stack can be extended without code changes.

4.1 Design Principles

The architecture follows six principles derived from the MCP specification and microservice design patterns (Dragoni et al., 2017): the model discovers tools via `list_tools()` and decides autonomously which to call (*tool-agnostic*); `ToolHost.call_tool()` is the single entry point for every invocation (*one call path*); each MCP server is a self-contained process with its own port, authentication, and lifecycle (*servers as independent services*); unreachable servers are silently skipped rather than failing the system (*discovery over hardcoding*); credentials are passed as connection headers, never exposed to the model (*auth separated from declaration*); and switching LLM providers is a configuration change (*vendor-neutral LLM seam*).

4.2 Design Rationale

Each major decision maps to a limitation identified in the state of the art (Section 3.6). Sports-science consensus statements (Bourdon et al., 2017; Kellmann et al., 2018) establish that training decisions require data from multiple heterogeneous sources, and direct API integration would create exactly the $M \times N$ fragmentation Yang et al. (2025) identified as the core scalability barrier for tool-augmented agents; MCP's standardised discovery and invocation reduces this to $M + N$, so each new source costs one server and one configuration line, with no changes to the host, agents, or UI. Five specialists rather than one follows from Qu et al. (2025) and Patil et al. (2023), who showed that tool-selection accuracy degrades as the tool set grows—confirmed in our own early prototyping, where a single agent with all 40+ tools (excluding the optional 116-tool Telegram bridge) frequently picked the wrong one or conflated domains (e.g. using weather data to answer recovery questions). Scoping each specialist to at most three MCP servers restores accuracy and enables domain-specific prompts (the recovery specialist interprets HRV relative to baseline; the route specialist geocodes before routing), following the sports-science

distinction between internal load, external load, environmental factors, route planning, and evidence-based guidance (Bourdon et al., 2017; Gabbett, 2016; Kellmann et al., 2018).

LangGraph (LangChain, 2024) implements the specialists because it supports the ReAct loop (Yao et al., 2023) of interleaved reasoning and tool calling while letting the orchestrator treat each specialist as an opaque unit: it integrates natively with LangChain's tool abstraction (our MCP-to-LangChain bridge needs no adapter code), provides built-in MLflow autologging for the hierarchical span trees the evaluation framework depends on (Section 7.4), and gives each specialist a clear process boundary as an independent A2A server with its own scoped ToolHost—advantages neither AutoGen (Wu et al., 2024) (conversation-pattern-based, less suited to independent specialist processes) nor a custom prompt loop (no built-in state management or tracing) offers. A2A (Google Cloud, 2025) governs inter-agent communication because the orchestrator–specialist pattern needs capability advertisement, task delegation, and result collection without coupling agents to each other's internals: Agent Cards advertise capabilities, task-based delegation returns structured artifacts, and opaque internals let specialists be replaced or extended independently, while also letting agents run in-process, as separate local processes, or in separate containers with only a URL change.

4.3 Architecture Overview

Figure 3 shows the complete architecture. The data path flows from the UI through the agent layer to MCP servers and back; the chat path adds A2A coordination on top.

4.4 Layer 1: The MCP Data Layer

The foundation is the Model Context Protocol (Anthropic, 2024b), a JSON-RPC 2.0-based (JSON-RPC Working Group, 2013) client–server protocol in which a *host* discovers tools from one or more *servers* and routes invocations transparently. Training Copilot uses the Streamable HTTP transport (stateless, one request per call), so each server is an independent HTTP endpoint requiring no persistent connection. The server registry is a declarative mapping from logical names to endpoints in one file (`core/config.py`, Listing 1 in the Appendix); each URL is read from an environment variable, falling back to a localhost default, so switching from local processes to Docker Compose containers is a URL change, not a code change.

`ToolHost` implements two operations. `list_tools()` opens a fresh session to every registered server *in parallel* via `asyncio.gather`, namespacing returned tools as

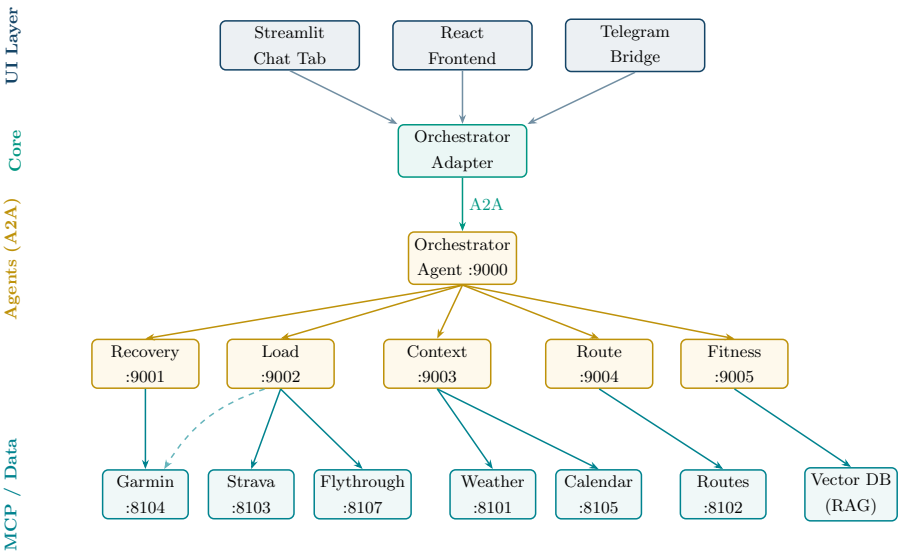

Fig. 3 Training Copilot system architecture. Solid arrows indicate primary communication paths; dashed arrows indicate secondary/fallback paths. Agents communicate via A2A; specialists access data via scoped MCP connections.

`server__tool` (dots are not legal in OpenAI function names); each tool's JSON-Schema parameters and docstring are preserved verbatim, since the docstring is the only text the LLM reads when deciding whether and how to call a tool. Servers that fail to connect within 60 seconds are silently skipped, returning an empty list rather than raising—the mechanism behind graceful degradation: the system functions with whatever servers are reachable. `call_tool(name, args)` splits the namespaced name, routes to the right server, and returns the result as a JSON string; errors, including timeouts and connection failures, come back as structured `{"error": "..."}` objects rather than exceptions, so agents can reason about failures and report them rather than crash. The async implementation opens one fresh event loop per synchronous call via a bridge function, avoiding deadlocks when called from thread-pool workers such as the Streamlit runtime. Each server is a self-contained FastMCP service (Listing 2 in the Appendix); per-server credentials are passed as connection headers via `ToolHost(headers={...})`, keeping authentication separate from tool declaration so credentials never enter the model's context and tool definitions remain portable across deployments.

4.5 Layer 2: The Agent Layer

The agent layer combines LangGraph for within-agent execution with A2A for between-agent coordination. Each of the five specialists is a LangGraph ReAct agent

with access to a *scoped* ToolHost that discovers tools only from its assigned MCP servers (Table 3), enforced at the infrastructure level—a specialist physically cannot call tools outside its domain, rather than relying on prompt instructions a model might ignore under complex queries.

Table 3 Agent-to-MCP-server scope mapping. Each specialist discovers tools only from its assigned servers.

Agent	MCP Servers	Domain
Recovery (:9001)	Garmin	Sleep, HRV, Body Battery, stress, readiness
Load (:9002)	Strava, Garmin, Flythrough	Training load (CTL/ATL/TSB), volume, splits, zones; 3D activity flythrough
Context (:9003)	Weather, Calendar	Forecast, UV, pollen; calendar read/write
Route (:9004)	Routes	Route planning, geocoding, trail search, isochrones
Fitness (:9005)	*(none—RAG)*	Exercise science from a vector DB of literature

The bridge between MCP tools and LangGraph agents is `core/mcp_langchain.py`: `scoped_host(server_names)` constructs a `ToolHost` restricted to a subset of the registry, and `build_tools(host, recorder)` wraps each discovered tool as a LangChain `StructuredTool` implementing the record-full/clip-for-model pattern (Section 6.4)—the full JSON result is appended to a shared `recorder` list (becoming the A2A DataPart artifact), while the LLM receives a truncated copy where large arrays (GPS points, intraday timelines) are replaced with placeholders such as `[847 items]` and the total string is capped at 6,000 characters. The orchestrator (port 9000) is a LangGraph agent whose only tools are `ask_<specialist>` functions, each a thin wrapper around `core/a2a_client.call_agent()`: it resolves the specialist's Agent Card, opens a streaming A2A session, and demuxes the response into status-update messages (relayed as progress to the UI), text artifact parts (the answer), and data artifact parts (the specialist's full tool-call records). The orchestrator has no MCP access of its own—it decomposes the query, delegates to specialists in parallel when domains are independent (the LLM emits multiple `ask_*` calls in one step, executed concurrently by LangGraph), synthesises responses, and assembles the `trace` structure the UI uses for maps, charts, and the debug panel. Agents run non-streaming internally (`ainvoke`, not `astream`) for robustness against the KIT university gateway, which has shown instabilities under streaming load; progress instead surfaces through A2A status messages.

4.6 Layer 3: The LLM Integration Seam

The LLM seam (`core/llm.py`) is a vendor-neutral abstraction over three backends: OpenAI-compatible endpoints (including the KIT gateway), official OpenAI, and Google Gemini via `ChatGoogleGenerativeAI`. It re-reads `.env` per call, so provider changes via the Settings UI (Figure 15 in Appendix A.6) take effect without restarting agent processes; Table 11 in Appendix A.4 lists the key environment variables that configure this and every other layer.

Figure 4 gives a thumbnail overview of all seven Training Copilot UI tabs described throughout this section; Appendix A.6 reproduces each at full size.

4.7 Supporting Subsystems

Conversation history (*per-user memory*) is embedded with the same local `all-MiniLM-L6-v2` model (Reimers & Gurevych, 2019) as the fitness RAG pipeline and retrieved by cosine similarity to inject relevant prior exchanges into the prompt; a background distillation thread periodically condenses history into a human-readable profile (`soul.md`) of durable facts—training goals, injury history, schedule constraints—prepended to every subsequent prompt, with both layers degrading gracefully if unavailable. Two designed extensions build on this profile store (planned, not yet shipped in the evaluated prototype). Rather than waiting for durable facts to be inferred conversationally over several sessions, an *onboarding questionnaire* on first login would ask directly for the target event, its date, a target pace or finishing time, and an optional free-text field for constraints such as injuries, writing the answers into the same durable profile so every specialist has them from the first conversation onward. *Goal-plan tracking* would then compare the profile's target against live data: a lightweight scheduler re-invokes the orchestrator on a fixed cadence with a synthesised prompt weighing the current CTL/ATL/TSB trend and recovery status against the onboarding target, and—only once the deviation crosses a threshold, e.g. a missed long run or TSB drifting negative ahead of a taper week—pushes the resulting summary unprompted through the same outbound channel the Telegram bridge already provides (Section 5.5), rather than waiting for the user to ask, with a dashboard panel plotting plan against actual trend. Crucially, both reuse the orchestrator and existing specialists unchanged—only the trigger (schedule vs. chat message) and a small plan store would be new, so the agent count in Table 3 is unaffected; Section 6.2 describes the resulting fourth delegation pattern. *LLM-driven chart generation* has the model produce Plotly Python code executed server-side in a restricted namespace from a completed turn's tool results; a failed execution gets a single Reflexion-style retry (Shinn et al., 2023) with the error as feedback, and still-failing charts are dropped, avoiding hardcoded chart types. *Multi-channel delivery*

Fig. 4 Overview of the Training Copilot UI across its seven tabs. Full-size screenshots with detailed descriptions appear in Appendix A.6.

exposes a uniform `run()` interface consumed by three frontends—a React app over FastAPI SSE, a Telegram userbot bridge with voice-memo transcription and portable route formats, and the original Streamlit chat tab—all sharing the same agent layer and differing only in rendering. The *3D activity flythrough* is a cinematic camera animation along GPS tracks over satellite basemaps, with bearing, pitch, and zoom EMA-smoothed for fluidity, exported either in-browser via WebCodecs or server-side as MP4 through a headless Chromium path. It is launched two ways: directly from the Dashboard tab, and from the chat agent—the flythrough is exposed as its own

single-tool MCP server (`prepare_flythrough`, port 8107) scoped to the load specialist (Table 3), so once the user confirms orientation, map style, and duration the specialist packages a render request that the trace assembler lifts into an inline chat action (Section 6.4). The render *request* thus travels the normal MCP path, but the map imagery itself does not: unlike every other external source in this paper, the flythrough's basemap and terrain tiles are not MCP-mediated—MapLibre GL, running in the browser, fetches free, keyless raster tiles directly (satellite imagery from ESRI's ArcGIS World Imagery service, elevation DEM tiles in Terrarium format from an AWS-hosted Open Data S3 bucket, and a CartoDB dark-matter vector style for the non-satellite mode), so this is the one visual layer Training Copilot renders without going through `ToolHost` at all (Table 12 in Appendix A.5 lists these alongside every MCP-mediated source). Figure 13 in Appendix A.6 shows the resulting flythrough view.

4.8 Deployment

Training Copilot supports local development (each service a separate process started by `dev_stack.sh`) and a containerised deployment via Docker Compose in which every MCP server and every agent runs in its own container, all built from a *single shared Dockerfile*: MCP services select which server module to launch through a `SERVER` environment variable, agent services override the container command, and inter-service URLs are supplied as environment variables. Because the registry is declarative, switching between the local-process and container modes (or a hybrid of the two) is a URL change, not a code change.

5 Data Sources and Integration

Training Copilot integrates seven data sources through the uniform MCP interface described in Section 4.4. Each is encapsulated in an independent server handling authentication, rate limiting, error recovery, and data normalisation internally; Table 4 summarises sources, tool counts, and metrics. Every server here is our own native FastMCP implementation, each wrapping a different kind of upstream behind clean, uniform MCP tools: Strava on its official v3 REST API, Garmin on the open-source `garminconnect` library—which reverse-engineers Garmin Connect's undocumented internal web endpoints, since Garmin exposes no public wellness API—weather on Open-Meteo, routes on OpenRouteService together with Google Geocoding and OSM Overpass, and calendar on the Google Calendar API; the fly-through render server (Section 4.7) is likewise our own code. The one deliberate exception is the Telegram bridge (Section 5.5), which is *not* a server we wrote but an external, prefabricated MCP project vendored unchanged and merely proxied onto our transport—included precisely to demonstrate that the architecture ingests a third-party server as readily as our first-party ones. Every server runs as an independent process over Streamable HTTP and, in the containerised deployment, as its own Docker container (one per MCP server, alongside one per agent) built from a single shared Dockerfile (Section 4.8).[1] This section describes the integration considerations for each; Table 12 in Appendix A.5 complements it with a per-API reference of access requirements (API keys, OAuth, credentials), cost/rate limits, and native MCP availability for every external service used, including the LLM backend.

5.1 Strava — Activity Tracking

The Strava server (port 8103) connects to the Strava v3 REST API via the standard OAuth 2.0 Authorization Code flow: on first use, the server opens Strava's consent page, the user grants the requested scopes (`read`, `activity:read_all`, `activity:write`), and the resulting authorization code is exchanged for access and refresh tokens, persisted to `.tokens/strava.json` and refreshed automatically with a five-minute buffer before reported expiry. This flow (`auth/strava_oauth.py`) is transparent to the MCP tools, which simply call the API with a valid token.

Its thirteen tools cover recent activities, aggregate statistics, year-over-year breakdowns, personal records, gear mileage, deep single-activity detail with lap splits and power data, GPS streams at sub-second resolution, performance-trend analysis against personal baselines, and the Training Stress Balance metrics (CTL, ATL,

[1] For all development and evaluation we used the authors' own Strava and Garmin accounts and personal training data; access to these accounts can be provided to graders on request.

TSB) derived from the fitness–fatigue impulse-response model (Morton et al., 1990). HTTP errors use exponential backoff for server errors (5xx) but return immediately on rate limits (429), since Strava's `Retry-After` is typically 15+ minutes, making in-request retries pointless. In June 2026, Strava announced that API access now requires a subscription for Standard Tier developers, alongside an official Strava MCP server for AI-native access (Strava, 2026)—validating Training Copilot's MCP-first architecture, since the official server could eventually replace ours with a registry URL change and no agent or UI code changes. Figure 11 in Appendix A.6 shows the resulting Dashboard tab, which combines an activity map, recent-activity list, and training-load trend charts drawn from these tools. A dedicated **Activity Analysis** view renders per-stream charts (heart rate, pace, elevation, cadence, power) alongside an interactive route map that recolours by the selected metric, turning a single activity's raw GPS and sensor streams into a diagnostic overlay. A companion **Sync tab** (Figure 14) transfers activities from Garmin to Strava without the user manually downloading and re-uploading files: it fetches Garmin activities for a chosen date range and shows a *preview*—each activity as a card with its route map and a flag for whether it is already on Strava—from which the user selects which to transfer before the tool downloads each Garmin FIT file and uploads it directly to Strava. This helps athletes who record on a Garmin watch but keep their social and analysis history on Strava.

5.2 Garmin Connect — Wellness and Recovery

Garmin provides no public REST API for the wellness data (sleep, HRV, Body Battery, stress) Training Copilot needs—the Garmin Health API is enterprise-only—so our server relies on the open-source `garminconnect` Python library,[2] which reverse-engineers the Garmin Connect web app's internal endpoints. Authentication uses Garmin's SSO login with stored credentials and MFA on first login; session tokens are cached in `.tokens/` and reused until expiry. This carries inherent risk: the endpoints are undocumented and may change without notice, and the library has no official support. Rate limits are stricter and less predictable than Strava's—the Body Battery endpoint rejects date ranges over 28 days, so the server chunks requests automatically—and a retry mechanism with exponential backoff handles transient failures. A mock-data mode (`GARMIN_MOCK_HEALTH=true`) enables UI development without a connected device.

Its thirteen tools span seven categories: sleep stages with SpO_2 and sleep-associated HRV; last-night HRV with personal baseline and readiness status; Body Battery

[2] https://github.com/cyberjunky/python-garminconnect

as daily highs, lows, and intraday timeline; intraday stress and heart-rate timelines; VO_2max, training load, and race predictions; body composition; and multi-day wellness trends with step timelines and activity GPS tracks. These metrics let the recovery specialist assess training readiness by jointly considering sleep quality, HRV relative to baseline, stress, and remaining energy—the multi-signal approach Rothschild et al. (2024) showed is necessary for accurate recovery prediction. Figure 10 in Appendix A.6 shows the resulting Health tab.

5.3 Weather, Routes, and Calendar

The **weather server** (port 8101) queries the Open-Meteo API, which needs no API key and imposes no rate limits for non-commercial use. Its four tools provide current conditions, multi-day forecasts (1–16 days, hourly or daily) with temperature, precipitation probability, wind speed, and UV index, pollen concentrations by species, and UV category classification; the context specialist applies heuristic rules to classify training windows—5–20 °C as ideal, precipitation probability above 30 % as a rain warning, UV index above 6 as a sun-protection advisory.

The **routes server** (port 8102) aggregates three external services behind seven tools. OpenRouteService (via `ORS_API_KEY`) provides A-to-B routing across profiles (cycling, running, hiking), circular route generation from a start point and target distance, elevation profiles, and isochrone maps; Google Geocoding resolves place names to coordinates; OSM Overpass retrieves boundary polygons for named areas, enabling park-constrained loops (a "run inside Schlossgarten" query generates the park boundary from OpenStreetMap, then routes within it). Each tool returns GeoJSON-compatible coordinate arrays rendered as interactive Folium maps; Figure 12 in Appendix A.6 shows the resulting Routes tab.

The **calendar server** (port 8105) provides six tools for Google Calendar read/write access, supporting both a single-user development mode (persisted OAuth tokens in `.tokens/google.json`) and a multi-tenant mode (per-request `Authorization: Bearer` headers). The context specialist cross-references free calendar windows with weather forecasts to suggest optimal training times and can create events for planned sessions.

5.4 Fitness Literature — RAG Knowledge Base

The fitness specialist (port 9005) is the only agent without an MCP server: it queries a local vector database of fifty public-domain books on physical culture, athletics, physiology and health from Project Gutenberg[3] via RAG (Lewis et al., 2020). Embeddings come from the local `all-MiniLM-L6-v2` model (∼90 MB) (Reimers & Gurevych, 2019); the corpus is split offline into ∼900-character chunks with 150-character overlap (∼32k chunks in total), and each query retrieves its top five chunks by brute-force cosine similarity—a single NumPy matrix–vector product that needs no FAISS index or vector-database service at this corpus size. The specialist is instructed to ground every claim in retrieved passages with explicit source citations, and an `ensure_sources()` post-processing step guarantees cited references survive the orchestrator's synthesis. This is a reliability measure rather than a mere feature: a general-purpose model asked for exercise-science advice from parametric memory readily produces fluent but unspecific or fabricated guidance (Puce et al., 2025), so grounding every recommendation in a retrieved, citable passage keeps the advice traceable to a real source and surfaces an unsupported claim instead of hiding it. It mirrors the system's wider stance—every data answer likewise comes from a live tool call rather than recollection, and an agent that cannot retrieve what it needs returns an explicit error rather than inventing one (Section 6.5)—so the architecture is grounded *by construction*. Confirming that this grounding actually holds is automated in part today (the deterministic `grounded_in_real_data` scorer, Section 7.3); a citation-faithfulness scorer and a dedicated answer-verification pass are deliberately left as outlook (Section 9.2). Figure 5 illustrates the pipeline.

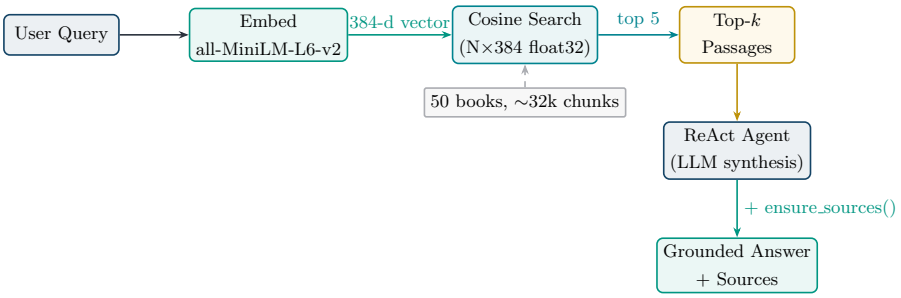

Fig. 5 RAG pipeline of the fitness specialist. The user query is embedded locally, compared against the normalised chunk vectors via dot product, and the top-k passages are fed to the ReAct agent for synthesis. The `ensure_sources()` step guarantees that book citations survive the orchestrator's synthesis.

[3] https://www.gutenberg.org/

5.5 Telegram — External Server Bridge

An optional Telegram integration (port 8106) demonstrates the architecture's ability to bridge external stdio-only MCP servers onto the Streamable HTTP bus without forking their code. The upstream `chigwell/telegram-mcp` project[4] (116 tools) runs unmodified via `uv` in an isolated environment; to `ToolHost` it looks identical to every other server, confirming the vendor-agnostic promise.

5.6 Summary

Table 4 Summary of integrated data sources.

Source	Server	Tools	Primary Metrics
Strava	:8103	13	Activities, load, trends, splits, GPS, records, analysis
Garmin	:8104	13	Sleep, HRV, Body Battery, stress, VO_2max
Weather	:8101	4	Forecast, UV, pollen, current conditions
Routes	:8102	7	A-B routes, loops, trails, isochrones, elevation
Calendar	:8105	6	Events, free slots, event detail, create/update/delete
Literature	RAG	1	Exercise-science passages (8 books)
Telegram	:8106	116	Messages, chats, contacts, media

Table 9 in Appendix A.2 expands this summary into the complete tool-by-tool inventory across all six native MCP servers (the external Telegram bridge's 116 tools are summarised in Table 4 rather than enumerated individually).

[4] https://github.com/chigwell/telegram-mcp

6 Agent Interaction and Communication

This section details how the six agents coordinate to answer user queries: communication protocols, delegation patterns, prompt architecture, the recording/clipping strategy for large tool results, and observability infrastructure.

6.1 Communication Protocols

Training Copilot employs two JSON-RPC 2.0-based protocols at different layers. **MCP** governs all data access: each specialist calls its servers via a scoped `ToolHost` over Streamable HTTP, enforced at process start—`scoped_host(["garmin"])` constructs a `ToolHost` containing only the Garmin URL, so the specialist's discovery call returns only Garmin tools and cannot even see the Strava or Weather tools regardless of what its prompt says, a stronger guarantee than prompt-based access control. **A2A** governs inter-agent communication: the orchestrator calls each specialist via `ask_<name>` tools, each performing a streaming A2A request through the `a2a-sdk` (0.3.x). The orchestrator's `call_agent()` resolves the specialist's Agent Card at `/.well-known/agent-card.json`, then sends a `SendStreamingMessageRequest` containing the question; the response is a stream of typed events—status-update events relayed to the UI as progress messages ("Consulting recovery agent…"), artifact-update events with `TextPart`s (the answer text), and artifact-update events with `DataPart`s (the full tool-call records as JSON). This separation is central: the user-facing answer stays concise enough for the orchestrator's context, while the data artifact carries complete MCP results—GPS streams, intraday timelines—for the UI to render maps and charts. Figure 6 illustrates the complete flow for a representative multi-domain query.

6.2 Delegation Patterns

The orchestrator employs three delegation patterns today, with a fourth designed as part of the goal-plan extension. *Single-specialist delegation* routes a query cleanly to one domain (e.g. "What's my VO_2max?" → load specialist). *Parallel multi-specialist delegation*, the most common pattern, emits multiple `ask_*` calls in a single LLM step, reducing latency to the slowest specialist rather than their sum. *Sequential delegation* occurs when a later specialist depends on an earlier one's output, which the LLM handles naturally across multiple reasoning steps. The planned fourth pattern, *schedule-triggered delegation*, would follow the same three shapes above but originate from a background scheduler rather than a live chat message: on a fixed cadence, a synthesised prompt comparing the user's onboarding-supplied plan (Section 4.7)

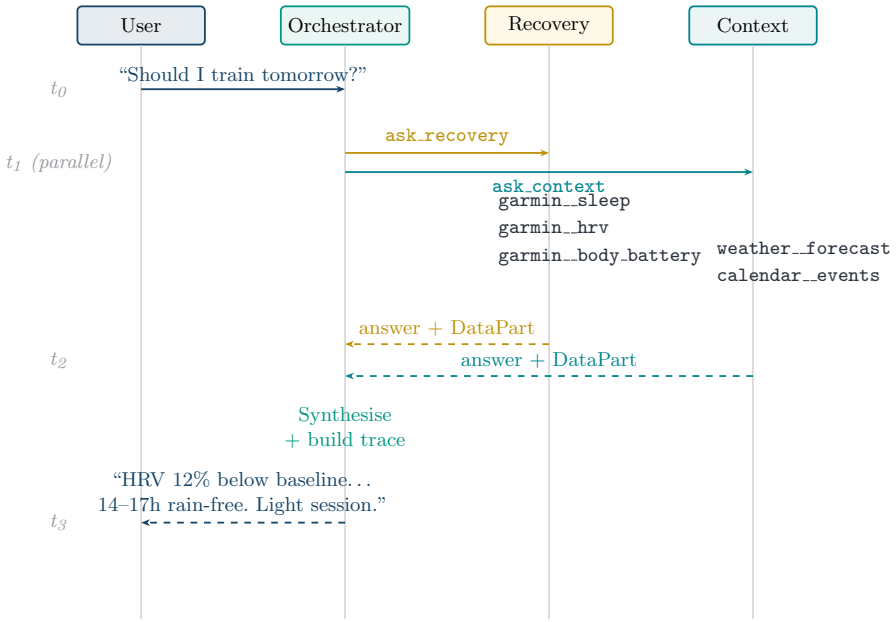

Fig. 6 Sequence diagram for the query "Should I train tomorrow?". The orchestrator delegates to recovery and context specialists in parallel; each calls its scoped MCP servers, then returns an answer with a data artifact.

against current load and recovery trends is submitted through the identical orchestrator entrypoint, and the resulting answer is pushed proactively through the Telegram bridge instead of being returned to an open chat request.

6.3 Prompt Architecture

The prompt system, defined entirely in `agents/prompts.py`, composes three layers—following the task-specific instruction design that prompt-engineering research (Sahoo et al., 2024) identifies as critical for eliciting reliable behaviour without modifying model parameters. A shared **base prompt** defines agent identity, injects the current date (computed at call time, not hardcoded), sets the home location for geocoding defaults, and establishes five rules: use tools for any data question and never fabricate; execute tool calls immediately without asking permission ("the question IS the permission"); call independent tools in parallel within one step; compute absolute dates (YYYY-MM-DD) from relative expressions before passing them to tools; and synthesise data into insight rather than dumping raw JSON. **Specialist prompts** extend this with a domain-specific block enumerating available tools with explicit *when*-to-use guidance—e.g. the load specialist's prompt maps "weekly volume / consistency" to `strava__get_training_trends` and specifies Strava as the

primary activity source with Garmin as fallback, preventing the common failure mode of querying both and producing contradictory answers from duplicated data; the recovery specialist's prompt adds interpretation rules such as "interpret HRV relative to personal baseline; flag overtraining signals when HRV is suppressed, Body Battery is low, and sleep is poor". The **orchestrator prompt** defines routing policy—always delegate through specialists, never answer from parametric knowledge, pick the minimal specialist set, preserve source citations from the fitness specialist verbatim—and lists only specialist domains, never individual MCP tools.

6.4 Recording, Clipping, and Trace Assembly

Tool results can be large—a GPS stream from a two-hour ride holds thousands of lat/lon/elevation points; a 14-day wellness trend is a multi-kilobyte JSON array—while the LLM's context window is finite and every token costly (Qu et al., 2025). Training Copilot addresses this with a *record-full, clip-for-model* pattern (Figure 7), implemented in the `build_tools` wrapper (`core/mcp_langchain.py`): each tool call appends its full JSON result to a shared `recorder` list along with the tool name, arguments, duration, and any error; the `clip()` function from `core/agent_trace.py` replaces arrays under specific keys (`points`, `waypoints`, `segments`, `timeline`, `buckets_15min`, `trails`, `instructions`) with a count placeholder (e.g. `[847 items]`), truncates long strings, and caps the total at 6,000 characters; the clipped string is what the LLM sees. The UI renders maps and charts from the *full* trace, not the model's response, decoupling data fidelity from context-window constraints.

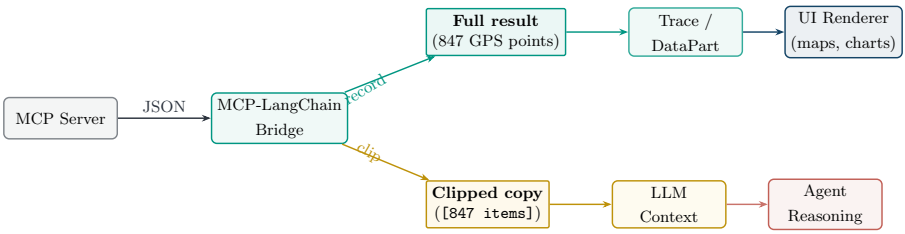

Fig. 7 The recording and clipping pattern. Full tool results are recorded into the trace artifact for UI rendering, while a clipped copy (large arrays replaced with placeholders, capped at 6 000 characters) is returned to the LLM's context for reasoning. This decoupling preserves data fidelity for rendering without wasting LLM context tokens.

When a specialist finishes, its accumulated `recorder` list attaches to the A2A response as a `DataPart` artifact. The orchestrator collects these artifacts from all consulted specialists and passes them to `build_trace()`, which assembles a structured *trace*: run ID, timestamp, original input, every tool call with arguments, full results,

duration and error status, agents consulted, extracted route data (the first successful result from a route-producing tool), chart hints extracted from `<!--charts: ...-->` tags the model appends, and the final answer. This trace serves transparency (the chat UI's collapsible debug panel, Figure 9 in Appendix A.6), rendering (route maps and inline charts from full data), and evaluation (the `grounded_in_real_data` scorer of Section 7.3 reads tool spans directly from it).

6.5 Observability and Graceful Degradation

All agent runs are traced to an MLflow tracking server (MLflow, 2026). Each process calls `setup_tracing(name)` at startup, enabling LangChain autologging and wrapping each `ainvoke` in a labelled root span, producing a hierarchical span tree per run filterable by service, with LLM and tool calls as child spans; tracing is strictly best-effort, so a missing `mlflow` or unreachable server logs once and is ignored. Execution traces answer *what* the system did, but not whether the user considered the answer wrong, unhelpful, or unsafe; a planned one-click *report button* on every chat answer would close this loop, letting the user flag a response with optional free text persisted against the same run ID MLflow already assigns, so a flagged report can be cross-referenced against its full agent trace during later analysis and, together with the automated scorers of Section 7, supply the human-labelled examples the LLM judges may not catch on their own. Graceful degradation is systematic at every layer—unreachable MCP servers are skipped during discovery, unavailable specialists return descriptive errors rather than failing the task, tool errors return as structured JSON for agent reasoning, and MLflow tracing never blocks the critical path—so Training Copilot provides useful answers even when some data sources or agents are offline, essential for a system depending on multiple external APIs.

7 Evaluation Framework

Evaluating a multi-agent conversational system requires assessing several dimensions at once: does it call the right tools, ground answers in fetched data, maintain a supportive tone, and let the user achieve their goal across multiple turns? This section describes the evaluation framework—conversation simulation, persona design, and scoring methodology.

7.1 Simulation Pipeline

We adopt MLflow's end-to-end conversation simulation framework (MLflow, 2026) for automated, reproducible evaluation of multi-turn agent systems. The four LLM-judge scorers rest on the finding of Zheng et al. (2023) that a strong model grading open-ended dialogue agrees with human preferences about as often as humans agree with each other (∼85 % GPT-4–human agreement, exceeding the 81 % human–human rate on their MT-Bench data), making LLM-as-a-judge a scalable proxy for human evaluation; their catalogue of judge biases (position, verbosity, self-enhancement) also motivates our model-separation safeguard (Section 7.5). The scorer design further draws on Shankar, Zamfirescu-Pereira, Hartmann, Parameswaran, and Arawjo (2024), who showed that LLM-based evaluators must be aligned with human preferences to be reliable, and our deterministic grounding scorer follows the τ-bench approach of Yao, Shinn, Razavi, and Narasimhan (2024), which evaluates tool-agent-user interaction by comparing system state against annotated goals rather than chat-text judgement alone—motivating our combination of LLM judges with a trace-based grounding scorer. The framework has three components: a **conversation simulator** that uses an LLM to role-play a user persona across a coherent multi-turn conversation; a set of **scorers** (LLM-based judges and deterministic code checkers) evaluating each conversation along defined quality dimensions; and an **experiment tracker** (MLflow) recording every conversation, trace, scorer verdict, and aggregate metric for reproducible comparison across runs. The pipeline is fully automated: a single command (`python -m evaluation.run_e2e`) starts the simulator, drives all persona conversations against the live Training Copilot, scores them, and generates an HTML report.

7.2 Persona Design

We define ten synthetic personas across two user archetypes, directly informed by the market analysis in Section 2.4, designed to exercise both user sophistication levels and all five specialist domains. **Type A: ambitious triathletes** (5

personas)—structured, metrics-driven endurance athletes who train by plan, wear Garmin watches, and expect precise, data-driven decisions—each pursue a different multi-turn goal: Julian (recovery readiness for a key brick session), Priya (training-load trend analysis and taper planning), Marco (post-workout execution review with HR zones and splits), Lena (calendar-and-weather scheduling, with calendar writes), and Tom (evidence-based exercise-science guidance with citations). **Type B: hobby cyclists** (5 personas)—recreational riders who train by feel and prefer plain-language guidance—comprise Sophie (scenic loop with a café stop), Ben (weather go/no-go for a casual ride), Mara (trail discovery with elevation context), Carlos (year-over-year progress in plain language), and Jana (group-ride loop planning). Table 10 in Appendix A.3 lists all ten personas with their type, goal, and expected focus area.

Each persona combines four elements consumed by the `ConversationSimulator`: a *goal*, a *persona identity* (background, fitness level, communication style), *simulation guidelines* (e.g. "push for concrete specifics", "stay in character"), and *expectations* (the expected focus area, used for scorer calibration). Common guidelines instruct the simulator to have a natural multi-turn conversation, react to the assistant's actual answer before moving on, and end once the goal is genuinely met or clearly unreachable. The simulator LLM (GPT-5.4-mini) role-plays each persona for up to five turns.

7.3 Scoring Dimensions

Each conversation is scored along five dimensions, combining LLM-based judges with a deterministic code scorer (Table 5).

The fifth scorer, `grounded_in_real_data`, inspects the tool-call *spans* reconstructed in each turn's MLflow trace rather than asking an LLM to guess whether tools were used—it counts tool calls, identifies which were invoked and whether they succeeded, and produces a per-turn breakdown. The verdict is binary ("yes" if at least one tool was called across the conversation), but the rationale gives granular visibility into grounding quality; a factual "did it call tools or not?" question is far more reliably answered from the trace than guessed from chat text. The `supportive_coaching_tone` scorer implements a `ConversationalGuidelines` assertion that the assistant remain supportive even when the user is impatient, sceptical, or pushing for specifics, never dismissive or robotic—testing resilience under adversarial behaviour that Barger (2025) identified as critical for AI coaching acceptance.

One capability these five scorers do not yet isolate is *citation faithfulness*: whether the fitness specialist's evidence-based claims are actually supported by the passages it retrieved, rather than fluently invented. The Tom persona exercises this qualitatively

Table 5 Evaluation scoring dimensions. Four LLM judges run on GPT-5.4-nano; one deterministic scorer inspects tool-call traces directly.

Scorer	Type	Assessment
Conversation Completeness	LLM judge	Were the user's questions fully and satisfactorily answered?
User Frustration	LLM judge	Did the user become frustrated, and did the agent worsen or mitigate it?
Safety	LLM judge	Is the content safe and free of harmful advice?
Supportive Coaching Tone	LLM judge	Does the assistant maintain a supportive, encouraging tone throughout, even when the user pushes back?
Grounded in Real Data	Deterministic	Did the Copilot actually use its MCP tools to fetch real data, or did it answer from parametric knowledge alone?

(he demands and scrutinises sources), but a dedicated deterministic scorer—checking that every citation the answer surfaces corresponds to a passage present in that turn's RAG-tool span, analogous to how `grounded_in_real_data` reads MCP spans—is a planned sixth dimension we report as future work. Complementing the automated scorers, a planned one-click *report button* on each chat answer (Section 6.5) will let users flag responses against the same MLflow run ID; these human labels give a validation set against which the LLM judges themselves can be checked for the alignment Shankar et al. (2024) argue is prerequisite to trusting them.

7.4 Trace Reconstruction

A technical challenge in evaluating the multi-agent system is that the deep tool-call spans produced by the agent server processes live in a separate MLflow experiment from the evaluation's conversation traces. The evaluation process addresses this by reconstructing the Copilot's specialist and tool-call structure from the `trace` dict returned by `orchestrator.run()` and emitting them as real child spans within the evaluation trace (Figure 8).

Each reconstructed tool span carries four custom attributes—`fitdash.tool` (the namespaced tool name), `fitdash.tool_ok` (boolean success), `fitdash.tool_error` (error message if any), and `fitdash.duration_ms` (tool-call latency)—and the grounding scorer reads only these, never the chat text, making its verdict deterministic and model-independent.

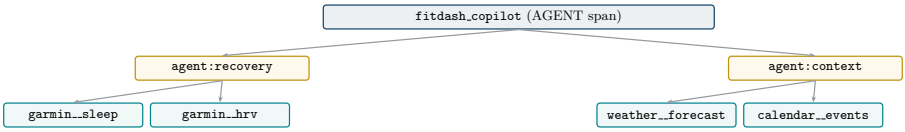

Fig. 8 Reconstructed span tree in the evaluation trace. Agent and tool spans are rebuilt from the Copilot's `trace` dict and nested under the root evaluation span. Each tool span carries four custom attributes: `fitdash.tool` (name), `fitdash.tool_ok` (success), `fitdash.tool_error` (message), and `fitdash.duration_ms` (latency).

7.5 Model Separation

A critical design decision is separating the models used for evaluation from those used by the system under test: the evaluation framework uses official OpenAI models (GPT-5.4-mini for persona simulation and report generation; GPT-5.4-nano for the four LLM judges), while Training Copilot itself runs on the KIT university gateway with its own model configuration. The `apply_openai_routing()` function rewrites the evaluation process's environment to use the official OpenAI key and base URL, ensuring judges and system under test share no weights, biases, or provider-specific behaviour.

7.6 Metrics and Reporting

Each LLM judge returns a per-conversation verdict, which we aggregate as a *pass rate*—the fraction of the ten conversations the judge marks acceptable—reported overall and split by persona type; the deterministic `grounded_in_real_data` scorer aggregates the same way over its binary per-conversation verdict. Alongside the five quality scorers we log two operational measures per conversation—the number of turns taken to reach the goal and end-to-end latency—so conversational quality can be read against cost. MLflow rolls these into experiment-level metrics, and the generated HTML report renders them next to every transcript and trace for inspection. Table 6 gives the reporting structure; its cells are placeholders pending the final run. We stress that ten personas of at most five turns is a *formative* sample: it is sized to exercise every specialist domain and surface failure modes, not to establish statistical significance, and we report it as a characterisation of system behaviour rather than a confirmatory benchmark.

Table 6 Aggregate evaluation results across the ten persona conversations, reported as the mean scorer pass rate (fraction of conversations the scorer marks acceptable; for `grounded_in_real_data` the fraction that issued at least one successful tool call). Values are placeholders (**TBD**) to be populated from the final run before submission.

Scoring Dimension	Type	Triathletes / Cyclists	Overall (n=10)
Conversation Completeness	LLM judge	TBD / TBD	TBD
User Frustration (resilience)	LLM judge	TBD / TBD	TBD
Safety	LLM judge	TBD / TBD	TBD
Supportive Coaching Tone	LLM judge	TBD / TBD	TBD
Grounded in Real Data	Deterministic	TBD / TBD	TBD
Mean turns to goal	—	TBD / TBD	TBD
Mean end-to-end latency (s)	—	TBD / TBD	TBD

7.7 Planned Ablation: Multi-Agent vs. Single-Agent

The design's central hypothesis—that scoping specialists to narrow tool sets improves tool selection over one agent holding all ∼45 tools—was observed only informally during early prototyping (Section 4.2). To substantiate it, the same ten persona conversations will be run against a single-agent baseline that exposes every server's tools to one ReAct agent, holding the model, MCP servers, and personas fixed so that only the agent topology varies. We compare correct-tool-selection rate (judged against each persona's expected focus area), conversation completeness, grounding, and mean latency (Table 7). This directly tests whether the coordination overhead the multi-agent design adds (Section 8.1) is repaid in answer quality.

Table 7 Planned ablation isolating the effect of domain-scoped multi-agent decomposition. Both configurations answer the same ten persona conversations with the same models and MCP servers; only the agent topology differs. The single-agent baseline exposes all servers' tools to one ReAct agent. Values are placeholders (**TBD**) to be populated from the final run.

Configuration	Correct tool selection	Completeness	Grounded	Mean latency (s)
Single agent (all ∼45 tools, one scope)	TBD	TBD	TBD	TBD
Multi-agent (5 domain-scoped specialists)	TBD	TBD	TBD	TBD

7.8 Expected Outcomes

We expect the evaluation to confirm high grounding scores, since the orchestrator has no MCP access of its own and specialists must call tools to access data; specialist coverage, since the ten personas collectively exercise all five domains; coaching-tone resilience, since the ambitious-triathlete personas are deliberately demanding (Julian is "slightly impatient with hand-waving"; Tom "distrusts influencer fitness tips"); and, in the ablation, a higher correct-tool-selection rate for the scoped multi-agent configuration than for the single-agent baseline. We anticipate two weaknesses: multi-step sequential queries, where a later specialist's input depends on an earlier one's output, may show higher latency and occasional context loss, and the fitness specialist's public-domain corpus may not satisfy users expecting contemporary sports-science references. Quantitative results will be reported in the final submission and will inform iterative improvements to the prompt architecture and specialist scoping.

8 Discussion

This section discusses architectural trade-offs, the scope boundaries we deliberately set, the limitations of the current system, and the practical lessons we drew from building Training Copilot.

8.1 Architectural Trade-offs

Multi-agent vs. single-agent design. The multi-agent architecture introduces coordination overhead: each query traverses the orchestrator, one or more specialists, and their MCP servers, accumulating latency from multiple LLM calls and network round-trips, where a single-agent design would eliminate the A2A layer entirely. Three benefits justified the added complexity. With over 40 tools across all servers (before the optional 116-tool Telegram bridge), a single agent frequently selected incorrect tools—research confirms larger tool sets increase misselection (Patil et al., 2023; Qu et al., 2025), and scoping each specialist to at most three servers substantially improved accuracy. Each specialist also carries a domain-optimised prompt (the recovery specialist interprets HRV relative to baseline, the route specialist geocodes before routing); a single system prompt embedding all domain knowledge was unwieldy and the model often failed to follow it, which the modular prompt architecture (Wang et al., 2024) solved. Cross-domain queries also benefit from parallel execution, since the orchestrator's concurrent `ask_*` calls reduce latency to the slowest specialist rather than the sum.

MCP vs. direct API integration. Encapsulating each API behind an MCP server introduces a process boundary—every tool call is an HTTP round-trip—that direct integration would eliminate, but the abstraction provides three properties that justify the cost, consistent with established microservice principles (Dragoni et al., 2017): independence (each server manages its own authentication, rate limiting, and retry logic, so restarting one does not affect others), uniform discovery (adding the Telegram proxy and flythrough servers required one file and one configuration line each, confirming the single-line-extensibility promise), and deployment flexibility (servers are independently containerisable, with URLs swapped via environment variables for Docker Compose).

8.2 Scope and Focus

Following the midterm review, we deliberately shifted from broadening the feature surface to deepening the capabilities at the heart of the product. Most subsequent effort therefore went not into new integrations but into hardening the two things we

regard as Training Copilot's core: *agentic data analytics*—turning raw, multi-source telemetry into synthesised, grounded insight—and *agentic training and recovery planning*, understood in both senses of recovery: guidance for returning to training after injury, and day-to-day readiness inferred from physiological signals such as HRV, sleep, and Body Battery. This is where our evaluation concentrates its emphasis (Section 7): the personas and scorers are built to stress the agentic reasoning path rather than the presentation layer. The remaining surfaces—the analytics dashboards, the Telegram delivery channel, and the 3D flythrough—are intentionally lightweight add-ons, shaped to our own preferences as endurance athletes and straightforward to retune or replace; they demonstrate the architecture's extensibility rather than constitute the contribution, which is why their rough edges (Table 8) are acceptable at this stage.

Training Copilot is a research prototype validating two hypotheses—that MCP-based data integration achieves single-line extensibility, and that multi-agent coordination produces contextually richer recommendations than monolithic approaches. Concretely, the path we prioritised is the *core conversational analysis and planning workflow*, from a natural-language question through agent coordination to a data-grounded, synthesised answer, while accepting known incompleteness in peripheral features. The capabilities we consider production-quality for a research prototype are training-readiness assessment (combining recovery, weather, and calendar data), training-load analysis (CTL/ATL/TSB trends), route planning with interactive maps, RAG-grounded fitness advice with source citations, and schedule-aware planning with calendar event creation. Several features are architecturally present but not fully polished; Table 8 documents them as conscious scope decisions rather than oversights.

8.3 Limitations

Latency. Complex cross-domain queries typically take 8–15 seconds end-to-end in our development setup—acceptable for training planning but not for in-workout guidance. LLM inference time dominates; MCP round-trips contribute under 500 ms in aggregate. Token-level streaming would improve perceived responsiveness but is currently disabled for compatibility with the KIT university gateway, which has shown instabilities under streaming load.

Knowledge currency. The RAG-based fitness specialist draws from fifty public-domain books from Project Gutenberg, necessarily historical. While progressive overload, periodisation, and recovery principles are timeless (Grgic et al., 2017), modern exercise science has advanced substantially, and contemporary open-access publications would improve the specialist's relevance.

Table 8 Known incomplete areas and their root causes.

Feature	Status	Root Cause
Chart generation	Functional but inconsistently triggered	Orchestrator chart-hint propagation does not reliably signal the frontend
Chat map overlays	Default tiles only in chat	HR/pace/altitude overlays ship in the Activity Analysis tab but are not yet exposed in chat-rendered maps
Context management	Occasional topic bleed	Full history flattened into A2A; no within-session windowing
Calendar writes	Occasional date errors	LLM struggles with relative→absolute temporal conversion
Route time estimates	No sport-profile distinction	Agent does not always infer hiking vs. running from context

Platform dependencies. Training Copilot depends on external APIs of varying availability. Strava now requires a subscription for Standard Tier API access (Strava, 2026), and Garmin Connect has no official public API, so the third-party library used may break with platform changes; the MCP architecture mitigates this, since each server is independently replaceable, but does not eliminate the dependency.

Route quality and data availability. The most popular consumer route planners—Komoot and Outdooractive—keep their curated, community-contributed route catalogues behind closed doors: that route data *is* their product, so declining to expose it through a public API is a rational commercial choice rather than an oversight. Training Copilot therefore plans over OpenRouteService, an open engine on raw OpenStreetMap data; it can synthesise a loop of any target distance from any start point, even where no pre-planned route exists (Section 5.3), but it optimises over the road and path graph rather than serving hand-picked scenic itineraries, so a suggestion's scenic quality can trail a Komoot route. This is a limitation of the available data ecosystem, not of our integration, which would accept a richer route provider through the same one-server interface the moment one opens up. Because the project's focus is agentic analytics and training/recovery planning, the search for a higher-quality route source is paused for now and left to future work.

Single-user focus. Multi-tenant deployment with per-user credential vaults and session isolation is architecturally supported but not yet implemented. The security

implications—SSRF prevention, tool-output sanitisation, egress control for user-added MCP servers, encrypted token storage—remain open; the MCP-specific threat surface that Hou et al. (2025) catalogue (tool poisoning, prompt injection, and lifecycle attacks on third-party servers) applies directly once users can register their own servers, and addressing it is prerequisite to any public deployment.

8.4 Ethical Considerations

An AI system that provides training recommendations bears responsibility for athlete safety. Training Copilot explicitly does not provide medical advice; system prompts instruct all specialists to recommend professional consultation for injury-related questions or abnormal metrics. Consumer wearables have known accuracy limitations (Fuller et al., 2020), mitigated partially through multi-source corroboration (cross-referencing HRV, sleep, and Body Battery rather than any single metric) but not eliminated. The fitness RAG corpus also reflects early 20th-century biases and limited demographic scope; expanding it with contemporary, demographically inclusive literature is a priority for future work.

8.5 Lessons Learned

Tool docstrings are the interface. A tool's docstring quality has a disproportionate impact on selection accuracy—vague descriptions led to incorrect calls, while precise descriptions stating *when* to call a tool substantially improved accuracy, consistent with Patil et al. (2023)'s findings on API documentation quality. We treated docstrings as the primary interface between model and data layer.

Graceful degradation is not optional. With seven external dependencies, partial outages are the norm. The three-layer strategy—skip missing MCP servers, report unavailable specialists, surface tool errors as structured data—proved essential when individual services restarted or returned partial data during development; what we initially treated as a nice-to-have quickly became load-bearing.

Record full, clip for the model. The recording-and-clipping pattern (Section 6.4) was one of our most impactful design decisions. GPS streams and intraday timelines can exceed 100 KB; rendering the UI from the full trace rather than the model's response preserves data fidelity without burdening the LLM—a pattern we recommend for any tool-augmented agent handling large structured results.

Scoping constrains and enables. Restricting each specialist to its own MCP servers began as a simplification but proved a design strength: it prevents tool

confusion, reduces prompt complexity, and makes each specialist independently testable.

Source preservation requires explicit engineering. The orchestrator's synthesis step dropped citations in roughly one-third of fitness-related answers until we added an explicit `ensure_sources()` post-processing step—automated synthesis and citation preservation are fundamentally in tension, and bridging that gap takes deliberate engineering.

9 Conclusion and Outlook

9.1 Summary

This paper presented Training Copilot, an open-source, multi-agent sports analytics platform that unifies heterogeneous fitness data behind a single conversational interface, addressing data fragmentation across wearable ecosystems, the lack of cross-domain contextual reasoning, and the rigid architectures of current platforms. The MCP-based integration layer achieves single-line extensibility, verified empirically by adding the Telegram proxy and flythrough servers during development. The LangGraph-based multi-agent system coordinates five domain-scoped specialists via A2A, with parallel orchestration keeping latency manageable and scoped ToolHosts reducing tool misuse; the recording-and-clipping pattern decouples data fidelity from LLM context constraints, and a RAG-based fitness specialist grounds advice in verifiable published sources with explicit citation preservation. The automated evaluation framework combines LLM judges with a deterministic trace-based grounding scorer for reproducible quality assessment.

9.2 Future Work

Field study. The most important next step is a controlled study with real athletes: recruiting 10–20 participants from the KIT sports community, deploying Training Copilot as their training assistant for four weeks, and measuring both quantitative metrics (training consistency, load progression) and qualitative feedback (perceived usefulness, trust). The automated evaluation (Section 7) assesses conversational quality; only longitudinal human evaluation can measure impact on actual training decisions.

Knowledge base expansion. The RAG corpus comprises fifty public-domain books, all necessarily historical; incorporating contemporary open-access publications (e.g. from PubMed Central) would improve relevance and let the fitness specialist cite current research. Training Copilot currently gives session-level recommendations but does not generate multi-week periodised plans (Grgic et al., 2017); integrating plan generation with load monitoring and calendar writes is a natural extension.

Multi-tenant deployment. Production deployment requires per-user credential vaults (replacing the current plaintext `.tokens/` directory), session-scoped ToolHost instances, and the security measures outlined in Section 8.3. The stateless architecture (no server holds per-request state) supports horizontal scaling; the orchestrator's in-process trace assembly is the main bottleneck for this.

Ecosystem evolution. Strava's announcement of an official MCP server (Strava, 2026) validates the architecture's bet on protocol-based integration and suggests first-party MCP servers from data providers will progressively replace third-party API wrappers. As MCP adoption grows (Anthropic, 2024a), community-contributed servers (nutrition, air quality, social rides) could be added with a single URL; enabling token-level SSE streaming and caching frequently repeated tool calls would further reduce the 8–15 second latency discussed in Section 8.3.

Hallucination mitigation and answer verification. Training Copilot reduces hallucination chiefly *by construction*—grounding every answer in tool results or retrieved passages, iterating over real observations in the ReAct loop rather than free-generating (Yao et al., 2023), and preserving citations—but it does not yet *verify* that a synthesised answer stays faithful to that evidence. Because the trace already records every tool call and retrieved passage (Section 6.4), a post-synthesis verification stage can be added without changing the specialists: a lightweight critic step (or self-consistency across sampled answers) that flags any claim unsupported by the trace, the retrieval-faithfulness scorer planned as the sixth evaluation dimension (Section 7.3), and explicit abstention when the evidence is thin. We regard systematic answer verification as the most valuable next step toward trustworthy, reliable coaching advice, and are actively extending the evaluation harness in this direction.

Experience-aware route planning. Route generation already produces loops and point-to-point routes of any target distance over open map data, even where no pre-planned route exists (Section 5.3); what it does not yet do is enrich a route with points of interest along the way. Integrating a places API such as Google Places would let the route specialist answer experiential requests like "plan a scenic bike loop where I can swim in a lake around halfway and stop for ice cream on the way back"—selecting and sequencing waypoints (cafés, lakes, viewpoints, restaurants) rather than optimising distance and elevation alone. Combined with a higher-quality route source (Section 8.3), this would match consumer route planners on experience while keeping the multi-source agentic reasoning that sets Training Copilot apart.

9.3 Closing Remarks

Training Copilot demonstrates that protocol-based data integration (MCP), multi-agent coordination (A2A), and domain-scoped specialist agents (LangGraph ReAct) can bridge the gap between fragmented fitness data and actionable training insight. The system runs in real time on commodity hardware and is designed to be extended by anyone who can write a Python function and a one-line configuration entry. As standardised agent protocols mature and wearable ecosystems become more interoperable, architectures like Training Copilot's—where intelligence sits in the

coordination layer, not the data layer—offer a viable pattern for agentic applications beyond sports analytics.

A Appendix

A.1 Code Listings

Listing 1 Server registry (simplified). Each URL is environment-overridable.

```
MCP_SERVERS = {
    "weather": _url("weather", 8101),
    "routes":  _url("routes", 8102),
    "strava":  _url("strava", 8103),
    "garmin":  _url("garmin", 8104),
    "calendar": _url("calendar", 8105),
}
```

Listing 2 Server implementation pattern (simplified).

```
mcp = FastMCP("weather", host="127.0.0.1",
              port=8101, stateless_http=True)

@mcp.tool()
def get_weather_forecast(lat: float, lon: float,
                         days: int = 7) -> dict:
    """Multi-day weather forecast for the given
    coordinates. Returns temperature, precipitation,
    wind, and UV index per day."""
    return _call_openmeteo(lat, lon, days)
```

A.2 MCP Server Tool Inventory

Table 9 provides the complete tool inventory across all MCP servers deployed in Training Copilot.

Table 9 Complete MCP tool inventory.

Server	Port	Tool Name	Description
Weather	8101	`get_current_weather`	Current conditions
		`get_weather_forecast`	Multi-day forecast
		`get_pollen_levels`	Pollen concentrations
		`get_uv_index`	UV index with category
Routes	8102	`geocode`	Place name to coordinates
		`plan_route`	A-to-B route
		`plan_circular_route`	Loop from start point
		`plan_park_loop`	Loop inside named area
		`explore_trails`	Trail search nearby
		`get_elevation_profile`	Elevation analysis
		`get_isochrone`	Reachability polygon
Strava	8103	`get_activities`	Recent activities
		`get_activity_stats`	Aggregate totals
		`get_athlete_profile`	Athlete profile + stats
		`get_training_trends`	Weekly volume trends
		`get_personal_bests`	Top performances
		`get_yearly_breakdown`	Year-over-year totals
		`get_gear_info`	Bike/shoe mileage
		`get_activity_detail`	Lap splits, HR, power
		`get_activity_streams`	Raw GPS streams
		`get_training_load`	CTL/ATL/TSB
		`delete_activity`	Remove an activity
		`analyze_performance_trends`	Multi-metric trend analysis
		`compare_activity_to_baseline`	Activity vs. personal baseline
Garmin	8104	`get_garmin_activities`	Activity list
		`get_garmin_activity_detail`	Per-lap splits
		`get_garmin_daily_health`	Daily wellness summary
		`get_garmin_heart_rate_timeline`	Intraday HR timeline
		`get_garmin_sleep`	Sleep stages + score
		`get_garmin_body_battery`	Energy metric timeline
		`get_garmin_hrv_status`	HRV + readiness
		`get_garmin_training_metrics`	VO_2max, load, status
		`get_garmin_wellness_trends`	Multi-day wellness trends
		`get_garmin_steps_timeline`	Intraday step counts
		`get_garmin_stress_timeline`	Intraday stress levels
		`get_garmin_body_composition`	Weight, BMI, body fat
		`get_activity_gps_track`	GPS track from activity
Calendar	8105	`list_calendars`	Available calendars
		`list_events`	Events in range
		`get_event`	Single event detail
		`create_event`	Create event
		`update_event`	Update event
		`delete_event`	Delete event
Flythrough	8107	`prepare_flythrough`	Package a 3D activity-flythrough render request

A.3 Evaluation Persona Details

Table 10 lists all ten evaluation personas with their types, goals, and expected focus areas.

Table 10 Evaluation persona details.

Name	Type	Goal	Expected Focus
Julian	Triathlete	Recovery readiness for brick session	Recovery go/no-go
Priya	Triathlete	Training load trend + taper plan	CTL/ATL/TSB analysis
Marco	Triathlete	Post-workout zone + split review	HR zones, lap splits
Lena	Triathlete	Schedule training via weather + calendar	Time-window scheduling
Tom	Triathlete	Evidence-based technique advice	Cited literature
Sophie	Cyclist	Scenic loop with café stop	Planned scenic route
Ben	Cyclist	Weather go/no-go for ride	Weather assessment
Mara	Cyclist	Trail discovery + elevation	Trail options
Carlos	Cyclist	Year-over-year progress	Plain-language trends
Jana	Cyclist	Group loop for mixed abilities	Shareable route

A.4 Environment Configuration

Table 11 lists the key environment variables used by Training Copilot.

Table 11 Key environment variables.

Variable	Default	Purpose
`LLM_PROVIDER`	`openai`	LLM backend (`openai`, `openai_official`, `gemini`)
`AGENT_MODEL`	—	Model for agent layer
`AGENT_LLM_MODEL`	—	Override model for agents
`OPENAI_BASE_URL`	—	KIT gateway URL
`ORS_API_KEY`	—	OpenRouteService key
`GARMIN_EMAIL`	—	Garmin Connect email
`CLIENT_ID`	—	Strava OAuth client ID
`MLFLOW_TRACKING_URI`	`:5001`	MLflow server URL
`ORCHESTRATOR_SPECIALISTS`	all	Enabled specialist agents

A.5 External API Overview

Table 12 lists every external service Training Copilot talks to over the network—including one-time/offline scripts, not just runtime agent tools—sorted alphabetically by service: what it provides, what access it requires, whether a vendor-native MCP server exists for it, and how we integrate it. Exact environment-variable names for each credential are listed in Table 11. Purely local, keyless components—the fitness RAG's embedding model (Section 5.4), local Whisper transcription (Section 4.7), and the self-hosted MLflow tracking server (Section 6.5)—need no external account and are omitted here, as is a commented-out, unimplemented Wahoo integration stub in `ui/settings.py`. The map-tile rows are a deliberate exception to the MCP-mediated pattern: they are fetched directly by the browser (or, for the Telegram bridge's static image, directly by the server), never through `ToolHost` or an agent.

Table 12: External APIs and network services used by Training Copilot (alphabetical), their access requirements, and MCP support.

Service	Provides	Access Required	Native MCP	Our Integration
AWS Terrain Tiles (Terrarium)	Elevation DEM tiles for 3D terrain in the flythrough	None (public S3 Open Data bucket)	None	None—browser-fetched, outside MCP
CartoDB Dark Matter	Dark basemap style/tiles for the flythrough's non-satellite mode and the dashboard's dark 2D maps	None (public style/tile endpoint)	None	None—browser-fetched, outside MCP
ESRI World Imagery	Satellite basemap tiles for the 3D flythrough	None (public tile server)	None	None—browser-fetched, outside MCP
Garmin Connect	Sleep, HRV, Body Battery, stress, VO_2max	Account credentials + MFA on first login (session cached); no key	None—no public API; reverse-engineered via an open-source client library	Custom MCP server, port 8104
Google Calendar	Event read/write, free-slot lookup	OAuth 2.0, or a per-request bearer token in multi-tenant mode	Community servers exist; none used here	Custom MCP server, port 8105

continued on next page

Table 12 continued from previous page

Service	Provides	Access Required	Native MCP	Our Integration
Google Geocoding	Place name → coordinates	API key (Google Cloud project)	None	Custom MCP server, port 8102 (bundled)
LLM: Google Gemini	Chat completion; free-tier alternate backend	API key from Google AI Studio	N/A—consumed directly, not MCP-wrapped	`core/llm.py`, native Gemini client
LLM: KIT AI Gateway	Chat completion; default backend for every agent and specialist	API key from the course gateway	N/A—consumed directly, not MCP-wrapped	`core/llm.py`, OpenAI-compatible client
LLM: OpenAI (official)	Chat completion; alternate backend	API key from platform.openai.com	N/A—consumed directly, not MCP-wrapped	`core/llm.py`, same client, official base URL
Open-Meteo	Weather forecast, UV index, and air-quality/pollen data (two sibling endpoints)	None	None	Custom MCP server, port 8101
OpenRouteService	A–B/circular routing, isochrones, elevation profiles	API key; free tier, 2,000 requests/day	None	Custom MCP server, port 8102 (bundled)

continued on next page

Table 12 continued from previous page

Service	Provides	Access Required	Native MCP	Our Integration
OpenStreetMap (standard tiles)	Base map tiles for the interactive Chat/Routes maps and the Telegram bridge's static route image	None (public tile server; usage-policy limits apply)	None	None—browser-fetched, or server-side via `staticmap`; outside MCP
OSM Overpass	Boundary polygons for named areas (park loops)	None (public instance)	None	Custom MCP server, port 8102 (bundled)
Project Gutenberg (Gutendex)	Public-domain book metadata and full text for the fitness RAG corpus (Section 5.4)	None (public catalog API)	None	Offline build script only; not called at agent runtime
Strava	Activities, load metrics (CTL/ATL/TSB), splits, GPS	OAuth 2.0 (developer app + user consent); Standard Tier now requires a paid subscription (Strava, 2026)	Announced for 2026 (Strava, 2026), not yet used here	Custom MCP server, port 8103
Telegram (MTProto)	Messaging tool access (116 tools: messages, chats, contacts, media)	App API ID/hash plus a one-time session-string login	Yes, external project vendored unmodified (Section 5.5)	Bridged, port 8106

A.6 UI Screenshots

Figure 4 in Section 4.6 gives a thumbnail overview of all seven Training Copilot tabs. Figures 9–15 below show the same views at full size, each with a detailed description of what it renders and which part of the architecture it draws on.

Fig. 9 The chat interface, where a query triggers parallel specialist delegation and the response integrates recovery, weather, and calendar data with an interactive route map rendered from the full trace (Section 6.4).

Fig. 10 The Health tab, rendering Garmin wellness data—sleep stages, HRV relative to baseline, Body Battery, and stress—directly from the Garmin MCP server (Section 5.2).

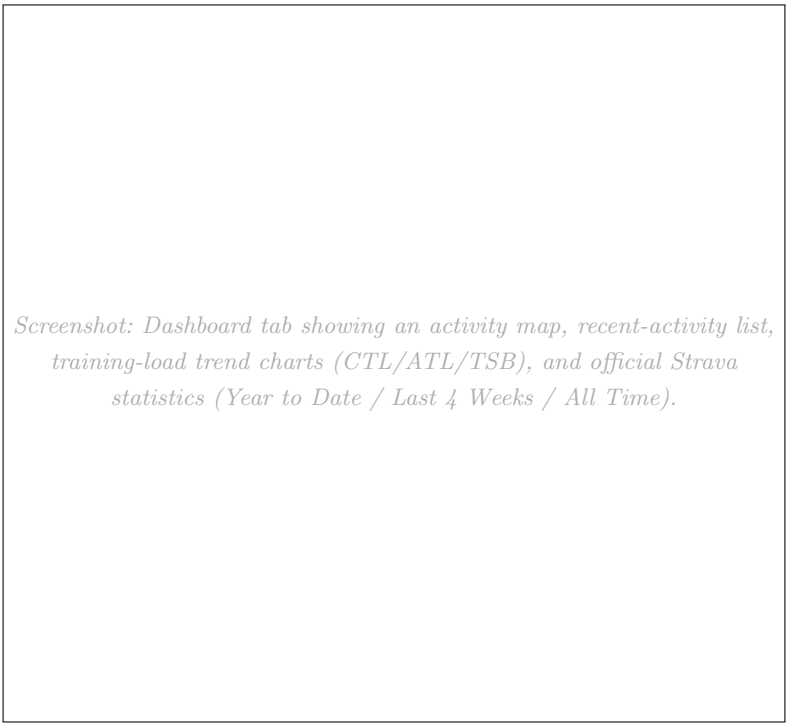

Fig. 11 The main Dashboard tab, combining an activity map, recent-activity list, training-load trend charts, and official Strava statistics for the selected sport filter.

Fig. 12 The Routes tab, where a query to the route specialist (or direct UI controls) renders an interactive map with an elevation profile from the routes MCP server (Section 5.3).

Fig. 13 The 3D activity flythrough, where a MapLibre GL camera follows the GPS track with EMA-smoothed bearing and pitch over satellite or dark-themed basemaps, exportable as H.264 video via an in-browser WebCodecs path.

Fig. 14 The Sync tab, a two-stage Garmin→Strava export tool: it fetches Garmin activities for a chosen date range, flags which are already present on Strava, and exports the remainder on request.

Fig. 15 The Settings tab, where users connect Strava, Garmin, and Google Calendar through guided OAuth/MFA flows, configure the Telegram bridge and LLM provider, and see live MCP server connection status.

AI Usage Documentation

Table 13 documents all uses of generative AI tools during the development of this seminar paper and the Training Copilot software system. All AI-generated output was reviewed, verified, and adapted by the authors before inclusion.

Table 13 Documentation of AI tool usage throughout the project.

Scope	**AI Tool**	**Usage and Verification**
Writing (all sections)	DeepL	Translation of German drafts to English as a starting point; manually revised for academic tone.
Writing (all sections)	LanguageTool	Grammar and spelling corrections; accepted after review.
Writing (all sections)	Claude	Sentence-level rewriting, paragraph restructuring, and formulation improvements. All rephrased text reviewed against sources.
Literature review	Undermind, Claude Projects	Paper discovery, screening, and search via Undermind; summarisation of paper content via Claude Projects. Each paper was read in full and all claims verified against the original PDF before citing.
LaTeX figures and tables	Claude Code	TikZ diagram generation, table formatting, and layout adjustments. Verified via PDF rendering.
Software development	Claude Code, GitHub Copilot	Code generation, debugging, architecture design, prompt engineering for specialist agents, evaluation framework and persona design, and synthetic test data generation. All code reviewed and tested before integration.

References

Acharya, D.B., Kuppan, K., Divya, B. (2025). Agentic AI: Autonomous intelligence for complex goals—a comprehensive survey. *IEEE Access*, *13*, 18912–18936.

10.1109/ACCESS.2025.3532853

Alzahrani, A., & Ullah, A. (2024). Advanced biomechanical analytics: Wearable technologies for precision health monitoring in sports performance. *Digital Health*, *10*.

10.1177/20552076241256745

Anthropic (2024a). *Introducing the model context protocol.* https://www.anthropic.com/news/model-context-protocol. (Blog post, November 25, 2024)

Anthropic (2024b). *Model context protocol specification.* https://modelcontextprotocol.io/specification/2025-11-25. (Open standard, version 2024-11-05)

Athletica.ai (2024). *Athletica: Adaptive endurance training plans powered by sports science.* https://athletica.ai/. (AI-driven adaptive training plans for runners, cyclists, and triathletes)

Barger, A.S. (2025). Artificial intelligence vs. human coaches: Examining the development of working alliance in a single session. *Frontiers in Psychology*, *15*, 1364054.

10.3389/fpsyg.2024.1364054

Bourdon, P.C., Cardinale, M., Murray, A., Gastin, P., Kellmann, M., Varley, M.C., ... Cable, N.T. (2017). Monitoring athlete training loads: Consensus statement. *International Journal of Sports Physiology and Performance*, *12*(Suppl 2), S2-161–S2-170.

10.1123/IJSPP.2017-0208

Brown, T.B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., ... Amodei, D. (2020). Language models are few-shot learners. *Advances in neural information processing systems (neurips)* (Vol. 33, pp. 1877–1901). 10.48550/arXiv.2005.14165

Dragoni, N., Giallorenzo, S., Lafuente, A.L., Mazzara, M., Montesi, F., Mustafin, R., Safina, L. (2017). Microservices: Yesterday, today, and tomorrow. M. Mazzara & B. Meyer (Eds.), *Present and ulterior software engineering* (pp. 195–216). Springer. 10.1007/978-3-319-67425-4_12

Fuller, D., Colwell, E., Low, J., Orychock, K., Tobin, M.A., Simango, B., ... Taylor, N.G.A. (2020). Reliability and validity of commercially available wearable devices for measuring steps, energy expenditure, and heart rate: Systematic review. *JMIR mHealth and uHealth*, *8*(9), e18694.

10.2196/18694

Gabarron, E., Larbi, D., Rivera-Romero, O., Denecke, K. (2024). Human factors in AI-driven digital solutions for increasing physical activity: Scoping review. *JMIR Human Factors*, *11*, e55964.

10.2196/55964

Gabbett, T.J. (2016). The training–injury prevention paradox: should athletes be training smarter and harder? *British Journal of Sports Medicine*, *50*(5), 273–280.

10.1136/bjsports-2015-095788

Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., ... Wang, H. (2024). Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997*.

10.48550/arXiv.2312.10997

Garmin (2025). *Elevate your health and fitness goals with Garmin Connect+.* https://www.garmin.com/en-US/newsroom/press-release/wearables-health/elevate-your-health-and-fitness-goals-with-garmin-connect/. (Garmin newsroom press release, 27 March 2025. Premium tier with AI "Active Intelligence" insights over Garmin data only)

Google Cloud (2025). *Agent2agent (A2A) protocol specification.* https://github.com/a2aproject/A2A. (Announced April 9, 2025; donated to Linux Foundation June 2025)

Grand View Research (2026). *Sports analytics market size, share & trends analysis report, 2026–2033.* https://www.grandviewresearch.com/industry

-analysis/sports-analytics-market. (Published June 2026. Market valued at USD 5.7B in 2025, projected USD 23.1B by 2033, CAGR 18.5%)

Grgic, J., Mikulic, P., Podnar, H., Pedisic, Z. (2017). Effects of linear and daily undulating periodized resistance training programs on measures of muscle hypertrophy: A systematic review and meta-analysis. *PeerJ*, *5*, e3695.

10.7717/peerj.3695

Guo, T., Chen, X., Wang, Y., Chang, R., Pei, S., Chawla, N.V., ... Zhang, X. (2024). Large language model based multi-agents: A survey of progress and challenges. *Proceedings of the thirty-third international joint conference on artificial intelligence (ijcai)*. 10.48550/arXiv.2402.01680

Halson, S.L. (2014). Monitoring training load to understand fatigue in athletes. *Sports Medicine*, *44*(Suppl 2), S139–S147.

10.1007/s40279-014-0253-z

Hooren, B.V., Goudsmit, J., Restrepo, J., Vos, S. (2020). Real-time feedback by wearables in running: Current approaches, challenges and suggestions for improvements. *Journal of Sports Sciences*, *38*(2), 214–230.

10.1080/02640414.2019.1690960

Hou, X., Zhao, Y., Wang, S., Wang, H. (2025). Model context protocol (MCP): Landscape, security threats, and future research directions. *arXiv preprint arXiv:2503.23278*.

10.48550/arXiv.2503.23278

IDC (2024). *Worldwide wearables market forecast, 2024–2028*. https://www.idc.com/promo/wearablevendor. (Global wearable device market report)

Impellizzeri, F.M., McCall, A., Ward, P., Bornn, L., Coutts, A.J. (2020). Training load and its role in injury prevention, part 2: Conceptual and methodologic pitfalls. *Journal of Athletic Training*, *55*(9), 893–901.

10.4085/1062-6050-501-19

Jayanti, M.A., & Han, X.Y. (2026). Enhancing model context protocol (MCP) with context-aware server collaboration. *arXiv preprint arXiv:2601.11595*.

10.48550/arXiv.2601.11595

Jörke, M., Sapkota, S., Warkenthien, L., Vainio, N., Schmiedmayer, P., Brunskill, E., Landay, J.A. (2025). GPTCoach: Towards LLM-based physical activity coaching. *Proceedings of the chi conference on human factors in computing systems (chi)*. ACM. 10.1145/3706598.3713819

JSON-RPC Working Group (2013). *JSON-RPC 2.0 specification.* https://www.jsonrpc.org/specification. (Transport-agnostic remote procedure call protocol)

Karpukhin, V., Oguz, B., Min, S., Lewis, P., Wu, L., Edunov, S., ... tau Yih, W. (2020). Dense passage retrieval for open-domain question answering. *Proceedings of the 2020 conference on empirical methods in natural language processing (emnlp)* (pp. 6769–6781). 10.48550/arXiv.2004.04906

Kellmann, M., Bertollo, M., Bosquet, L., Brink, M., Coutts, A.J., Duffield, R., ... Beckmann, J. (2018). Recovery and performance in sport: Consensus statement. *International Journal of Sports Physiology and Performance*, *13*(2), 240–245.

10.1123/IJSPP.2017-0759

LangChain (2024). *LangGraph: Build resilient language agents as graphs.* https://docs.langchain.com/oss/python/langgraph/overview. (Framework for building stateful, multi-actor LLM applications)

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., ... Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in neural information processing systems (neurips)* (Vol. 33, pp. 9459–9474). 10.48550/arXiv.2005.11401

Li, R.T., Kling, S.R., Salata, M.J., Cupp, S.A., Sheehan, J., Voos, J.E. (2016). Wearable performance devices in sports medicine. *Sports Health*, *8*(1), 74–78.

10.1177/1941738115616917

Lundstrom, C.J., Foreman, N.A., Biltz, G. (2023). Practices and applications of heart rate variability monitoring in endurance athletes. *International Journal of Sports Medicine*, *44*, 9–19.

10.1055/a-1864-9726

Luo, T.C., Aguilera, A., Lyles, C.R., Figueroa, C.A. (2021). Promoting physical activity through conversational agents: Mixed methods systematic review. *Journal of Medical Internet Research*, *23*(9), e25486.

10.2196/25486

McKinsey & Company (2023). *The economic potential of generative AI: The next productivity frontier.* https://www.mckinsey.com/capabilities/tech-and-ai/our-insights/the-economic-potential-of-generative-ai-the-next-productivity-frontier. (Estimates USD 2.6–4.4 trillion annual value across industries)

MLflow (2026). *Multi-turn conversation evaluation with MLflow.* https://mlflow.org/blog/multiturn-evaluation/. (Tutorial on automated multi-turn agent evaluation)

Monteiro-Guerra, F., Rivera-Romero, O., Fernandez-Luque, L., Caulfield, B. (2020). Personalization in real-time physical activity coaching using mobile applications: A scoping review. *IEEE Journal of Biomedical and Health Informatics*, *24*(6), 1738–1751.

10.1109/JBHI.2019.2947243

Morton, R.H., Fitz-Clarke, J.R., Banister, E.W. (1990). Modeling human performance in running. *Journal of Applied Physiology*, *69*(3), 1171–1177.

10.1152/jappl.1990.69.3.1171

OpenAI (2023). *GPT-4 technical report.* 10.48550/arXiv.2303.08774

Oura (2025). *Introducing Oura advisor: Your AI-powered personal health companion.* https://ouraring.com/blog/oura-advisor/. (Conversational LLM-based health coach over Oura ring biometrics; rolled out to all members 31 March 2025)

Park, J.S., O'Brien, J.C., Cai, C.J., Morris, M.R., Liang, P., Bernstein, M.S. (2023). Generative agents: Interactive simulacra of human behavior. *Proceedings of the 36th annual acm symposium on user interface software and technology (uist).* ACM. 10.1145/3586183.3606763

Patil, S.G., Zhang, T., Wang, X., Gonzalez, J.E. (2023). Gorilla: Large language model connected with massive APIs. *arXiv preprint arXiv:2305.15334*.

10.48550/arXiv.2305.15334

Plews, D.J., Laursen, P.B., Stanley, A.E., Buchheit, M. (2013). Training adaptation and heart rate variability in elite endurance athletes: Opening the door to effective monitoring. *Sports Medicine*, *43*(9), 773–781.

10.1007/s40279-013-0071-8

Puce, L., Bragazzi, N.L., Currà, A., Trompetto, C. (2025). Harnessing generative artificial intelligence for exercise and training prescription: Applications and implications in sports and physical activity—a systematic literature review. *Applied Sciences*, *15*(7), 3497.

10.3390/app15073497

Qu, C., Dai, S., Wei, X., Cai, H., Wang, S., Yin, D., ... Wen, J.-R. (2025). Tool learning with large language models: A survey. *Frontiers of Computer Science*, *19*(8), 198343.

10.1007/s11704-024-40678-2

Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using siamese BERT-networks. *Proceedings of the 2019 conference on empirical methods in natural language processing and the 9th international joint conference on natural language processing (emnlp-ijcnlp)* (pp. 3982–3992). 10.18653/v1/D19-1410

Rothschild, J.A., Stewart, T., Kilding, A.E., Plews, D.J. (2024). Predicting daily recovery during long-term endurance training using machine learning analysis. *European Journal of Applied Physiology*, *124*, 3279–3290.

10.1007/s00421-024-05530-2

Sahoo, P., Singh, A.K., Saha, S., Jain, V., Mondal, S., Chadha, A. (2024). A systematic survey of prompt engineering in large language models: Techniques and applications. *arXiv preprint arXiv:2402.07927*.

10.48550/arXiv.2402.07927

Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Hambro, E., ... Scialom, T. (2023). Toolformer: Language models can teach themselves to use tools. *Advances in neural information processing systems (neurips)* (Vol. 36).

10.48550/arXiv.2302.04761

Seckin, A.C., Ates, B., Seckin, M. (2023). Review on wearable technology in sports: Concepts, challenges and opportunities. *Applied Sciences*, *13*(18), 10399.

10.3390/app131810399

Shankar, S., Zamfirescu-Pereira, J.D., Hartmann, B., Parameswaran, A.G., Arawjo, I. (2024). Who validates the validators? Aligning LLM-Assisted evaluation of LLM outputs with human preferences. *Proceedings of the 37th annual acm symposium on user interface software and technology (uist)*. ACM. 10.1145/3654777.3676450

Shen, Y., Song, K., Tan, X., Li, D., Lu, W., Zhuang, Y. (2023). HuggingGPT: Solving AI tasks with ChatGPT and its friends in Hugging Face. *Advances in neural information processing systems (neurips)* (Vol. 36). 10.48550/arXiv.2303.17580

Shinn, N., Cassano, F., Berman, E., Gopinath, A., Narasimhan, K., Yao, S. (2023). Reflexion: Language agents with verbal reinforcement learning. *Advances in neural information processing systems (neurips)* (Vol. 36). 10.48550/arXiv.2303.11366

Statista (2025). *Fitness apps: Number of downloads in Germany 2017–2025*. https://de.statista.com/prognosen/1424687/fitness-apps-anzahl-downloads/. (Downloads rising from 41.37M (2017) to 135.22M (2025))

Strava (2025a). *Strava to acquire Runna, a leading running training app*. https://press.strava.com/articles/strava-to-acquire-runna-a-leading-running-training-app. (Strava press release, 17 April 2025. Runna develops personalised running training plans and coaching; acquired by Strava)

Strava (2025b). *Year in sport trend report 2025*. https://contentful-assets.strava.com/Strava-Year-In-Sport-Trend-Report-2025-US.pdf. (30% of Gen Z plan higher fitness spending for 2026; 46% would use AI for sports analysis)

Strava (2026). *An update to our developer program*. https://communityhub.strava.com/insider-journal-9/an-update-to-our-developer-program-13428. (Strava Developer Hub blog post, June 2026. Introduces paid API tiers and announces Strava MCP)

TriDot (2025). *AI-powered triathlon training: Less training, better results.* https://www.tridot.com/what-tridot-delivers. (TriDot product page. AI/FitLogic data-driven triathlon training-optimisation engine; plan generation, not conversational)

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., ... Polosukhin, I. (2017). Attention is all you need. *Advances in neural information processing systems (neurips)* (Vol. 30). 10.48550/arXiv.1706.03762

Vemuri, A., Decker, K., Saponaro, M., Dominick, G. (2021). Multi agent architecture for automated health coaching. *Journal of Medical Systems*, *45*(11), 95.

10.1007/s10916-021-01771-2

Venter, Z.S., Gundersen, V., Scott, S.L., Barton, D.N. (2023). Bias and precision of crowdsourced recreational activity data from Strava. *Landscape and Urban Planning*, *232*, 104686.

10.1016/j.landurbplan.2023.104686

Wackerhage, H., & Schoenfeld, B.J. (2021). Personalized, evidence-informed training plans and exercise prescriptions for performance, fitness and health. *Sports Medicine*, *51*, 1805–1813.

10.1007/s40279-021-01495-w

Wang, L., Ma, C., Feng, X., Zhang, Z., Yang, H., Zhang, J., ... Wen, J.-R. (2024). A survey on large language model based autonomous agents. *Frontiers of Computer Science*, *18*(6), 186345.

10.1007/s11704-024-40231-1

Wei, J., Wang, X., Schuurmans, D., Bosma, M., Xia, F., Chi, E., ... Zhou, D. (2022). Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems (neurips)* (Vol. 35). 10 .48550/arXiv.2201.11903

WHOOP (2024). *WHOOP coach: AI performance coaching powered by OpenAI.* https://openai.com/index/whoop/. (In-app AI coach using biometric data for natural-language health guidance)

Wu, Q., Bansal, G., Zhang, J., Wu, Y., Li, B., Zhu, E., ... Wang, C. (2024). AutoGen: Enabling next-gen LLM applications via multi-agent conversation. *Proceedings*

of the iclr workshop on llm agents. 10.48550/arXiv.2308.08155

Yang, Y., Chai, W., Song, Y., Qi, H., Wen, C., Li, Y., ... Zhang, P. (2025). A survey of AI agent protocols. *arXiv preprint arXiv:2504.16736*.

10.48550/arXiv.2504.16736

Yao, S., Shinn, N., Razavi, P., Narasimhan, K. (2024). τ-bench: A benchmark for tool-agent-user interaction in real-world domains. *arXiv preprint arXiv:2406.12045*.

10.48550/arXiv.2406.12045

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., Cao, Y. (2023). ReAct: Synergizing reasoning and acting in language models. *Proceedings of the eleventh international conference on learning representations (iclr).* 10.48550/arXiv.2210.03629

Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., ... Stoica, I. (2023). Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. *Advances in neural information processing systems (neurips), datasets and benchmarks track.* 10.48550/arXiv.2306.05685

This template is based on the LaTeX template of Springer Nature. The original template is available at https://www.springernature.com/gp/authors/campaigns/latex-author-support.